

RH134

Improving Command-line Productivity



Red Hat | Enterprise Linux

Goal

- Run commands more efficiently by using advanced features of the Bash shell, shell scripts, and various utilities provided by Red Hat Enterprise Linux.

Objectives

- Automate sequences of commands by writing a simple shell script.
- Efficiently run commands over lists of items in a script or from the command-line using for loops and conditionals.
- Find text matching a pattern in log files and command output using the **grep** command and regular expressions.

What is the script?

- A script is nothing more than a list of system commands stored in an executable file. At the very least, this saves the effort of retyping that particular sequence of commands each time it is invoked.

What is the script?

- A script is nothing more than a list of system commands stored in an executable file. At the very least, this saves the effort of retyping that particular sequence of commands each time it is invoked.

Why shell scripting?

- A working knowledge of shell scripting is essential to anyone wishing to become reasonably proficient at system administration.
- Consider that as a Linux machine boots up, it executes the shell scripts in `/etc/rc.d` to restore the system configuration and set up services.
- The craft of scripting is not hard to master, since the scripts can be built in bite-sized sections and there is only a fairly small set of shell-specific operators and options to learn
- The syntax is simple and straightforward, similar to that of invoking and chaining together utilities at the command line.

When not to use shell scripts?

- Resource-intensive tasks, especially where speed is a factor (sorting, hashing, recursion ...).
- Procedures involving heavy-duty math operations, especially floating-point arithmetic, arbitrary precision calculations, or complex numbers (use C++ or FORTRAN instead).
- Need native support for multi-dimensional arrays.
- Need to generate/ manipulate graphics or GUIs.
- Need direct access to system hardware.
- Need to use libraries or interface with legacy code.

If any of the above applies, consider a more powerful scripting language -- perhaps Perl, Python, Ruby or possibly a compiled language such as C, C++, or Java.

Useful tips for writing effective bash scripts in Linux

- Always use comments in scripts.
- Make a script exit when fails.
- Make a script exit when bash uses undeclared variable.
- Use double quotes to reference variables.
- Use functions in scripts.
- Always perform debugging for long scripts.
- Make your functions and variable names more descriptive.



What is a sha-bang?

- The sha-bang (**#!**) at the head of a script tells your system that this file is a set of commands to be fed to the command interpreter indicated.
- Immediately following the sha-bang is a path name. This is the path to the program that interprets the commands in the script.
- This command interpreter then executes the commands in the script, starting at the top (line following the sha-bang line), ignoring comments.
- Note that the path given at the "sha-bang" must be correct, otherwise an error message -- usually "**Command not found.**" -- will be the only result of running the script.
- The default shell is **bash** in a Linux system – try "**echo \$SHELL**" to display your current shell.

What is a sha-bang?

- This command interpreter then executes the commands in the script, starting at the top (line following the sha-bang line), ignoring comments.
- Note that the path given at the "sha-bang" must be correct, otherwise an error message -- usually "**Command not found.**" -- will be the only result of running the script.
- The default shell is **bash** in a Linux system – try "**echo \$SHELL**" to display your current shell.
- **#!** can be omitted if the script consists only of a set of generic system commands, using no internal shell directives.

```
#!/bin/sh  
#!/bin/bash  
#!/usr/bin/csh  
#!/usr/bin/perl  
#!/usr/bin/python
```

Invoking the script

- Having written the script, you can invoke it by "**sh scriptname**" or alternatively "**bash scriptname**".
- But not recommended is using `sh <scriptname`, since this effectively disables reading from stdin within the script.
- Much more convenient is to make the script itself directly executable with a **chmod**.

`chmod 555 scriptname` (gives everyone read/execute permission).

`chmod +rx scriptname` (gives everyone read/execute permission).

`chmod u+rx scriptname` (gives only the script owner read/execute permission).

- Having made the script executable, you may now test it by **./scriptname**.
- If it begins with a "sha-bang" line, invoking the script calls the correct command interpreter to run it.

Invoking the script

`chmod 555 scriptname` (gives everyone read/execute permission).

`chmod +rx scriptname` (gives everyone read/execute permission).

`chmod u+rx scriptname` (gives only the script owner read/execute permission).

- Having made the script executable, you may now test it by `./scriptname`.
- If it begins with a "sha-bang" line, invoking the script calls the correct command interpreter to run it.
- As a final step, after testing and debugging, you would likely want to move it to `/usr/local/bin` (as root, of course), to make the script available to yourself and all other users as a systemwide executable. The script could then be invoked by simply typing `scriptname` [ENTER] from the command line.

Quoting special characters

- A number of characters and words have special meaning to the Bash shell. However, occasionally you will want to use these characters for their literal values, rather than for their special meanings. To do this, use one of three tools to remove (or escape) the special meaning: **the backslash** (\), **single quotes** ('), or **double quotes** (").
- The backslash escape character removes the special meaning of the single character immediately following it. For example, to display the literal string **# not a comment** with the **echo** command, the # sign must not be interpreted by Bash as having special meaning. Place the backslash character in front of the # sign.

```
[user@host ~]$ echo # not a comment
```

```
[user@host ~]$ echo \# not a comment  
# not a comment
```


- When you need to escape more than one character in a text string, either use the escape character multiple times or employ single quotes ('). Single quotes preserve the literal meaning of all characters they enclose. Observe the escape character and single quotes in action:

```
[user@host ~]$ echo # not a comment #
```

```
[user@host ~]$ echo \# not a comment #  
# not a comment
```

```
[user@host ~]$ echo \# not a comment \#  
# not a comment #
```

```
[user@host ~]$ echo '# not a comment #'  
# not a comment #
```

- **"dot"**, as a component of a filename. When working with filenames, a leading dot is the prefix of a "hidden" file, a file that a ls will not normally show.

```
[root@server ~]# touch .hidden-file
```

- When considering directory names, a single dot represents the current working directory, and two dots denote the parent directory.

```
[root@server ~]# pwd  
/home/karim/projects  
[root@server ~]# cd ..  
[root@server ~]# pwd  
/home/karim
```


- The dot often appears as the destination (directory) of a file movement command, in this context meaning current directory.

```
[root@server ~]# cp /home/karim/projects/* .
```

- "dot" character match. When matching characters, as part of a regular expression, a "dot" matches a single character.
- ``` command substitution. The ``command`` construct makes available the output of command for assignment to a variable. This is also known as backquotes or backticks.

```
[root@Lab01 ~]# DATE=`date` == DATE=$(date)
[root@Lab01 ~]# echo $DATE
Tue Dec 22 21:35:27 EET 2020
```

- **!** It also inverts the meaning of a test operator. This can, for example, change the sense of equal (=) to not-equal (!=). The ! operator is a Bash keyword.
- ***** wild card [asterisk]. The * character serves as a "wild card" for filename expansion in globbing. By itself, it matches every filename in a given directory
The * also represents any number (or zero) characters in a regular expression.
- In the context of arithmetic operations, the * denotes multiplication.
- **?** wild card. The ? character serves as a single-character "wild card" for filename expansion in globbing, as well as representing one character in an extended regular expression.

- **\$** Variable substitution (contents of a variable).
- **\$** end-of-line. In a regular expression, a "\$" addresses the end of a line of text.
- **\$?** exit status variable. The \$? variable holds the exit status of a command, a function, or of the script itself.
- **\$\$** process ID variable. The \$\$ variable holds the process ID of the script in which it appears.
- **()** command group. (a=hello; echo \$a)
- **()** array initialization. Array=(element1 element2 element3)
- **{xxx,yyy,zzz,...}** Brace expansion.

```
[root@Lab01 karim]# touch {file1,file2,file3,file4,file5}
[root@Lab01 karim]# ls
file1 file2 file3 file4 file5
[root@Lab01 karim]#
```

- **{a..z}** Extended Brace expansion.

```
[root@Lab01 karim]# echo {a..z}
a b c d e f g h i j k l m n o p q r s t u v w x y z
[root@Lab01 karim]# echo {1..20}
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
[root@Lab01 karim]#
```

- **{ } \;** pathname. Mostly used in find constructs.

```
[root@Lab01 karim]# ls
file1 file2 file3 file4 file5
[root@Lab01 karim]# find . -name file1 -exec rm -f {} \;
[root@Lab01 karim]# ls
file2 file3 file4 file5
[root@Lab01 karim]#
```

- **[]** Test expression between [].
- **[[]]** Test expression between [[]]. More flexible than the single-bracket [] test.

```
[root@Lab01 karim]# x=5;y=10
[root@Lab01 karim]# [[ $x -lt $y ]] && echo $y
10
[root@Lab01 karim]# [[ $x -gt $y ]] && echo $y
[root@Lab01 karim]#
```


➤ **> &> >& >> <** redirection.

scriptname **>** filename redirects the output of scriptname to file filename.

Overwrite filename if it already exists.

command **&>** filename redirects both the stdout and the stderr of command to filename.

command **2>&1** redirects stdout of command to stderr.

scriptname **>>** filename appends the output of scriptname to file filename.

If filename does not already exist, it is created.

➤ **|** [pipe] Passes the output (stdout of a previous command to the input (stdin) of the next one, or to the shell.

```
[root@Lab01 karim]# echo ls -l
ls -l
[root@Lab01 karim]# echo ls -l | sh
total 0
-rw-r--r-- 1 root root 0 Dec 22 21:51 file2
-rw-r--r-- 1 root root 0 Dec 22 21:51 file3
-rw-r--r-- 1 root root 0 Dec 22 21:51 file4
-rw-r--r-- 1 root root 0 Dec 22 21:51 file5
```

```
[root@Lab01 karim]# ls -l | tr 'a-z' 'A-Z'
TOTAL 0
-RW-R--R-- 1 R00T R00T 0 DEC 22 21:51 FILE2
-RW-R--R-- 1 R00T R00T 0 DEC 22 21:51 FILE3
-RW-R--R-- 1 R00T R00T 0 DEC 22 21:51 FILE4
-RW-R--R-- 1 R00T R00T 0 DEC 22 21:51 FILE5
```

Exit and exit status

- The **exit** command may be used to terminate a script, just as in a C program. It can also return a value, which is available to the script's parent process.
- Every command returns an exit status (sometimes referred to as a return status or exit code). A successful command returns a **0**, while an unsuccessful one returns a **non-zero** value that usually may be interpreted as an error code.
- When a script ends with an exit that has no parameter, the exit status of the script is the exit status of the last command executed in the script (previous to the exit).

```
#!/bin/bash

COMMAND_1

. . .

# Will exit with status of last command.
COMMAND_LAST

exit
```

```
#!/bin/bash

COMMAND_1

. . .

# Will exit with status of last command.
COMMAND_LAST

exit $?
```


Testing script Inputs

Every reasonably complete programming language can test for a condition, then act according to the result of the test. Bash has the test command, various bracket and parenthesis operators, and the if/then construct.

Test constructs

- An **if/ then** construct tests whether the exit status of a list of commands is 0 (since 0 means "success" by UNIX convention), and if so, executes one or more commands.
- There exists a dedicated command called **[** (left bracket special character). It is a synonym for **test**, and a built-in for efficiency reasons. This command considers its arguments as comparison expressions or file tests and returns an exit status corresponding to the result of the comparison (0 for true, 1 for false).
- Bash introduced the **[[...]]** extended test command, which performs comparisons in a manner more familiar to programmers from other languages. Note that **[[** is a keyword, not a command.

Bash sees `[[ $a -lt $b ]]` as a single element, which returns an exit status.

```
[root@Lab01 karim]# x=5;y=10
[root@Lab01 karim]# [[ $x -lt $y ]] && echo $?
0
[root@Lab01 karim]# [[ $y -lt $x ]] || echo $?
1
```


Conditional structures

➤ There are various types of conditional statements in Bash

- if-else statement.
- if..elif..else statement.
- Nested if/ then condition.

▪ if-else statement

```
if [ condition-true ]
then
    command 1
    command 2
    ...
else
    # Optional (may be left out if not needed).
    # Adds default code block executing if original condition tests false.
    command 3
    command 4
    ...
fi
```

▪ if..elif..else statement

- ▣ **elif** is a contraction for else if. The effect is to nest an inner if/then construct within an outer one.

```
if [ condition1 ]
then
    command1
    command2
    command3
elif [ condition2 ]
# Same as else if
then
    command4
    command5
else
    default-command
fi
```


■ Nested if/ then Condition

- Condition tests using the if/then construct may be nested. The net result is equivalent to using the && compound comparison operator above.

```
if [ condition1 ]  
then  
  if [ condition2 ]  
  then  
    do-something # But only if both "condition1" and "condition2" valid.  
  fi  
fi
```

- When **if** and then are on same line in a condition test, a **semicolon** must terminate the if statement. Both if and then are keywords. Keywords (or commands) begin statements, and before a new statement on the same line begins, the old one must terminate.

```
if [ -x "$filename" ]; then
```

Regular expressions

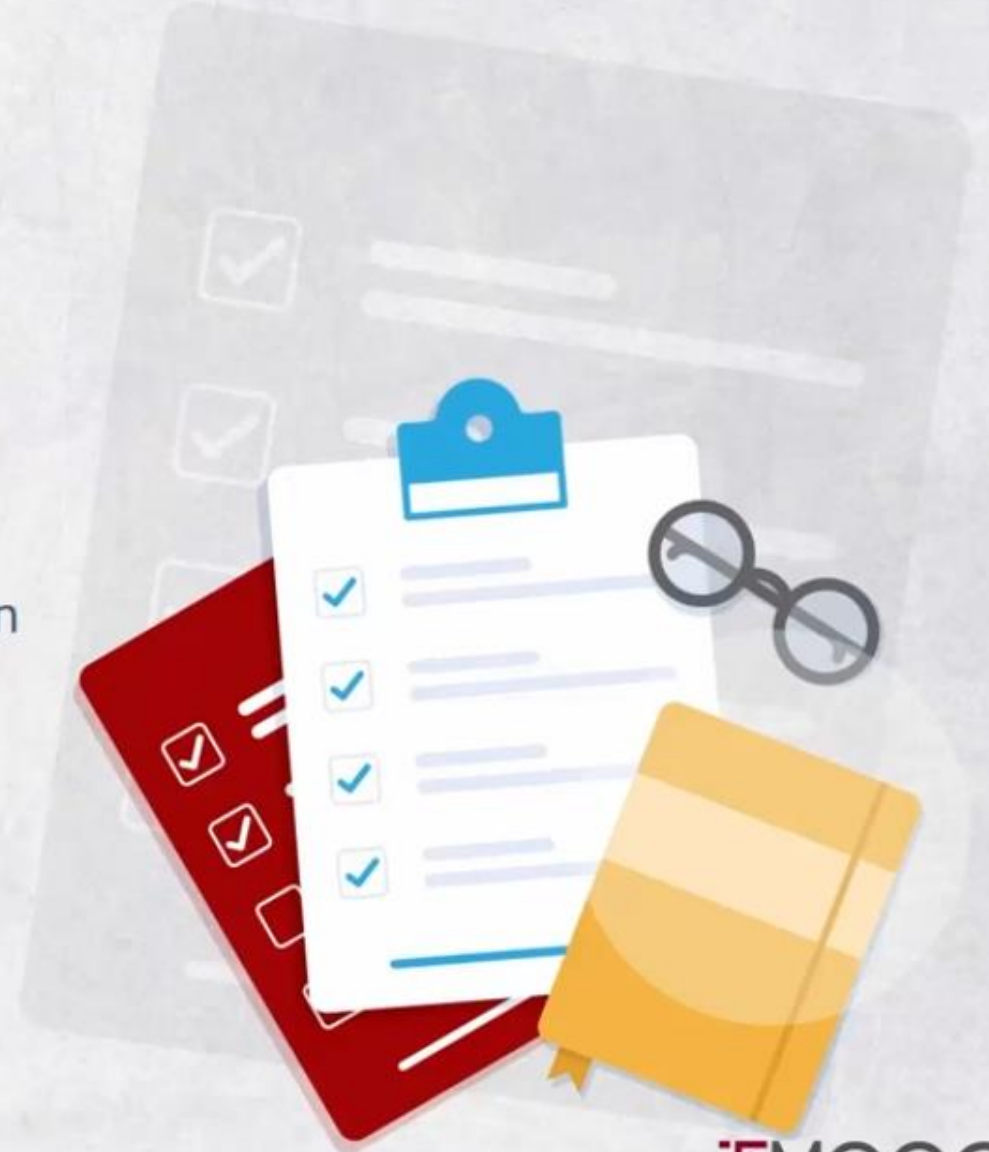
- Some of the commonly used commands with Regular expressions are tr, sed, vi and grep. Listed below are some of the basic Regex.

Symbol	Descriptions
.	replaces any character
^	matches start of string
\$	matches end of string
*	matches up zero or more times the preceding character
\	Represent special characters
()	Groups regular expressions
?	Matches up exactly one character

Summary

In this chapter, you learned:

- How to create and execute simple Bash scripts.
- How to use loops to iterate through a list of items from the command-line and in a shell script.
- How to search for text in log files and configuration files using regular expressions and grep.



RH134

Scheduling Future Tasks



Red Hat | Enterprise Linux

Goal

- Schedule tasks to automatically execute in the future.

Objectives

- Set up a command that runs once at some point in the future.
- Schedule commands to run on a repeating schedule using a user's crontab file.
- Schedule commands to run on a repeating schedule using the system crontab file and directories.
- Enable and disable systemd timers, and configure a timer that manages temporary files.

Scheduling Future Linux Tasks



Scheduling one-time tasks with at command

- Use the `at` TIMESPEC command to schedule a new job. The **at** command then reads the commands to execute from the stdin channel. While manually entering commands, you can finish your input by pressing Ctrl+D. For more complex commands that are prone to typographical errors, it is often easier to use input redirection from a script file, for example, **at now +5min < myscript**, rather than typing all the commands manually in a terminal window.
- The TIMESPEC argument with the **at** command accepts many powerful combinations, allowing users to describe exactly when a job should run. Typically, they start with a time, for example, **02:00pm**, **15:59**, or even **teatime**, followed by an optional date or number of days in the future.
- The following lists some examples of combinations that can be used.
 - **now +5min**
 - **teatime tomorrow (teatime is 16:00)**
 - **noon +4 days**
 - **5pm august 3 2021**

Inspecting and managing deferred user jobs

- To get an overview of the pending jobs for the current user, use the command **atq** or the **at -l** commands.

```
[user@host ~]$ atq
```

```
❶ 28 ❷ Mon Feb  2 05:13:00 2015 ❸a ❹user  
29  Mon Feb  3 16:00:00 2014 h user  
27  Tue Feb  4 12:00:00 2014 a user
```

In the preceding output, every line represents a different job scheduled to run in the future.

- ❶ The unique job number for this job.
- ❷ The execution date and time for the scheduled job.
- ❸ Indicates that the job is scheduled with the default queue a. Different jobs may be scheduled with different queues.
- ❹ The owner of the job (and the user that the job will run as).

Create at task

- **at +time** (to make a schedule for some commands to run in this time then enter.
- Time like now,1hour,6AM,tomorrow,4PM+3days , 10AM Jul 31, teatime ---> 4 pm).
- **at>** write the commands you want to execute in this time each command in one line then CTRL +d ---> to finish.
- **atq** -----> show you all schedules created.
- **atrm + job** -----> to delete the schedule.
- **Time can be:**
 - HH:MM 23:15
 - Midnight 12:00 am
 - Noon 12:00 pm

At Job Important Files

- **/etc/at.allow** (the file which has the users who can use at commands).
- **/etc/at.deny** (the file which has the users who denied from using at commands).
- **/var/spool/at/** (Holds the jobs script files).

Notes

- If the both files not exists that is mean all users are denied.
- The default installation create **at.deny** and make as default all users allowed to use at commands so take care.

Describing recurring user jobs

- Jobs scheduled to run repeatedly are called recurring jobs. Red Hat Enterprise Linux systems ship with the **crond** daemon, provided by the **crontab** package, enabled and started by default specifically for recurring jobs. The **crond** daemon reads multiple configuration files: one per user (edited with the **crontab** command), and a set of system-wide files. These configuration files give users and administrators fine-grained control over when their recurring jobs should be executed.
- Fields in the **crontab** file appear in the following order:
 - Minutes.
 - Hours.
 - Day of month.
 - Month.
 - Day of week.
 - Command.

Create crontab

```
# For details see man 4 crontabs

# Example of job definition:
# ----- minute (0 - 59)
# | ----- hour (0 - 23)
# | | ----- day of month (1 - 31)
# | | | ----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | ----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
```

- **crontab -e** (to create a cron , you should enter the full path of the command which you want to execute it) which + command (to get the full path of the command).
- ***/3** (in the minutes field means every 3 minutes , in the hours field --> means every 3 hours ...etc).
- **crontab -e -u +user** (to edit in cron of specific user).
- **crontab -l** (to list cron scheduled).
- **crontab -l -u +user** (to list the crons of specific user).
- **crontab -r** (to remove your cron).

Scheduling user jobs

- Normal users can use the **crontab** command to manage their jobs. This command can be called in four different ways:

Crontab Examples

COMMAND	INTENDED USE
crontab -l	List the jobs for the current user.
crontab -r	Remove all jobs for the current user.
crontab -e	Edit jobs for the current user.
crontab <i>filename</i>	Remove all jobs, and replace with the jobs read from <i>filename</i> . If no file is specified, stdin is used.

Example recurring user jobs

- The following job executes the command **/usr/local/bin/yearly_backup** at exactly 9 a.m. on February 2nd, every year.

```
0 9 2 2 * /usr/local/bin/yearly_backup
```

- The following job sends an email containing the word **Chime** to the owner of this job, every five minutes between 9 a.m. and 5 p.m., on every Friday in July.

```
*/5 9-17 * Jul 5 echo "Chime"
```

- The following job runs the command **/usr/local/bin/daily_report** every weekday at two minutes before midnight.

```
58 23 * * 1-5 /usr/local/bin/daily_report
```


Example recurring user jobs

- The following job executes the **mutt** command to send the mail message **Checking in** to the recipient **boss@example.com** on every workday (Monday to Friday), at 9 a.m.

```
0 9 * * 1-5 mutt -s "Checking in" boss@example.com % Hi there boss, just checking in.
```

Crontab examples

0	8	*	*	*	Once_a_day
0,30	9	*	*	*	Twice_a_day
0,30	8-18	*	*	*	Twenty_Two_times_a_day
*/5	*	*	*	*	Every_five_minutes
12	13	1	*	*	Once_a_month
49	23	16	9	*	Once_a_year
0	15	*	*	1	Every_monday
32	14	1	1	1	??? (caveat!)

Cron Job Important Files

- **/etc/cron.d** (This is the path to put (your) cron files created).
- **/etc/cron.deny** (the file to put who users you want to deny them from creating cron).
- **/etc/cron.allow** (to put the users who allowed to create cron).
- **/var/spool/cron** (it contains all the crons of all users, so you can access any file you want).
- **/etc/crontab** (to see the syntax for writing a cron task).
- **/etc/anacrontab** (its a file contain all scripts runs in specific time , if you cat this file you will see all scripts which will run in future and time ...etc).
- **/etc/cron.daily** (list scripts which will be executed daily).
- **/etc/cron.monthly** (list scripts which will be executed every month).

Managing temporary files

- Some applications (and users) use the **/tmp** directory to hold temporary data, Red Hat Enterprise Linux 7 and later include a new tool called **systemd-tmpfiles**, which provides a structured and configurable method to manage temporary directories and files.
- When systemd starts a system, one of the first service units launched is **systemd-tmpfiles** setup. This service runs the command **systemd-tmpfiles --create --remove**. This command reads configuration files from **/usr/lib/tmpfiles.d/*.conf**, **/run/tmpfiles.d/*.conf**, and **/etc/tmpfiles.d/*.conf**. Any files and directories marked for deletion in those configuration files is removed, and any files and directories marked for creation (or permission fixes) will be created with the correct permissions if necessary.
- To ensure that long-running systems do not fill up their disks with stale data, a **systemd** timer unit called **systemd-tmpfiles-clean.timer** triggers **systemd-tmpfiles-clean** service on a regular interval, which executes the **systemd-tmpfiles --clean** command.

Managing temporary files

- Some applications (and users) use the **/tmp** directory to hold temporary data, Red Hat Enterprise Linux 7 and later include a new tool called **systemd-tmpfiles**, which provides a structured and configurable method to manage temporary directories and files.
- When systemd starts a system, one of the first service units launched is **systemd-tmpfiles** setup. This service runs the command **systemd-tmpfiles --create --remove**. This command reads configuration files from **/usr/lib/tmpfiles.d/*.conf**, **/run/tmpfiles.d/*.conf**, and **/etc/tmpfiles.d/*.conf**. Any files and directories marked for deletion in those configuration files is removed, and any files and directories marked for creation (or permission fixes) will be created with the correct permissions if necessary.
- To ensure that long-running systems do not fill up their disks with stale data, a **systemd** timer unit called **systemd-tmpfiles-clean.timer** triggers **systemd-tmpfiles-clean** service on a regular interval, which executes the **systemd-tmpfiles --clean** command.

Cleaning temporary files manually

- Use the following **systemctl** command to view the contents of the **systemd-tmpfiles-clean.timer** unit configuration file.
- [user@host ~]\$ **systemctl cat systemd-tmpfiles-clean.timer.**
- In the preceding configuration the parameter **OnBootSec=15min** indicates that the service unit called **systemd-tmpfiles-clean.service** gets triggered 15 minutes after the system has booted up. The parameter **OnUnitActiveSec=1d** indicates that any further trigger to the **systemd-tmpfiles-clean.service** happens 24 hours after the service unit was activated last.

Cleaning Temporary Files Manually

- Based on your requirement, you can change the parameters in the **systemd-tmpfiles-clean.timer** timer unit configuration file. For example, the value **30min** for the parameter **OnUnitActiveSec** triggers the systemd-tmpfiles-clean.service service unit 30 minutes after the service unit was last activated. As a result, systemd-tmpfiles-clean.service gets triggered every 30 minutes after bringing the changes into effect.
- After changing the timer unit configuration file, use the **systemctl daemon-reload** command to ensure that systemd is aware of the change. This command reloads the systemd manager configuration.

```
[root@host ~]# systemctl daemon-reload
```

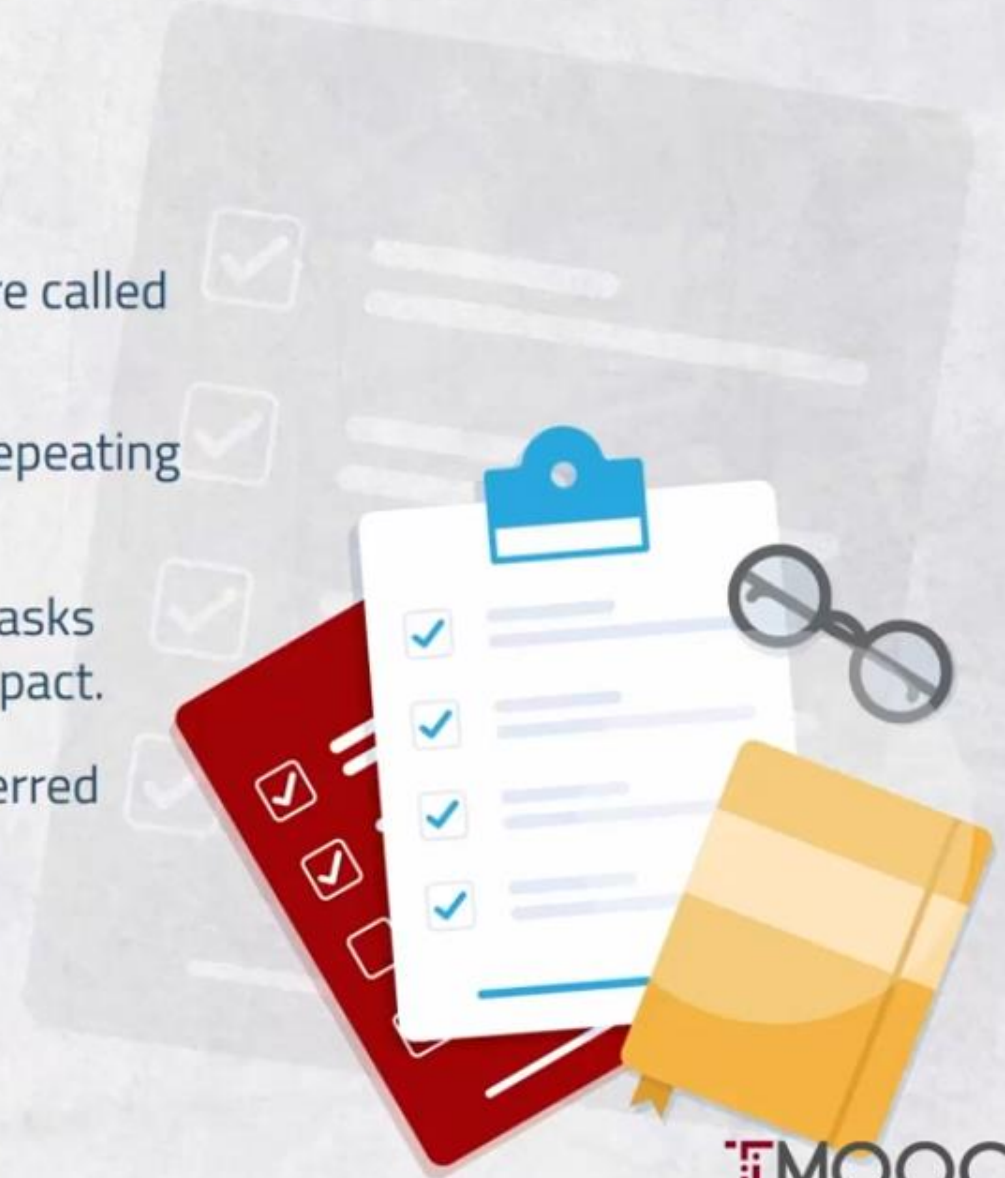
Configuration file precedence

- Configuration files can exist in three places:
 - **/etc/tmpfiles.d/*.conf**
 - **/run/tmpfiles.d/*.conf**
 - **/usr/lib/tmpfiles.d/*.conf**
- The files in **/usr/lib/tmpfiles.d/** are provided by the relevant RPM packages, and you should not edit these files.
- The files under **/run/tmpfiles.d/** are themselves volatile files, normally used by daemons to manage their own runtime temporary files.
- The files under **/etc/ tmpfiles.d/** are meant for administrators to configure custom temporary locations, and to override vendor-provided defaults.

Summary

In this chapter, you learned:

- Jobs that are scheduled to run once in the future are called deferred jobs or tasks.
- Recurring user jobs execute the user's tasks on a repeating schedule.
- Recurring system jobs accomplish administrative tasks on a repeating schedule that have system-wide impact.
- The **systemd** timer units can execute both the deferred or recurring jobs.



RH134

Tuning System Performance



Red Hat | Enterprise Linux

Goal

- Improve system performance by setting tuning parameters and adjusting scheduling priority of processes.

Objectives

- Optimize system performance by selecting a tuning profile managed by the tuned daemon.
- Prioritize or de-prioritize specific processes with the nice and renice commands.

Tuning systems

- System administrators can optimize the performance of a system by adjusting various device settings based on a variety of use case workloads. The tuned daemon applies tuning adjustments both statically and dynamically, using tuning profiles that reflect particular workload requirements.
- **Configuring Static Tuning**
Static tuning configures predefined kernel parameters in profiles that **tuned** applies at runtime. With static tuning, kernel parameters are set for overall performance expectations, and are not adjusted as activity levels change.
- **Configuring Dynamic Tuning**
With dynamic tuning, the **tuned** daemon monitors system activity and adjusts settings depending on runtime behavior changes. Dynamic tuning is continuously adjusting tuning to fit the current workload, starting with the initial settings declared in the chosen tuning profile.

➤ **INSTALLING AND ENABLING TUNED**

To install and enable the package manually:

```
[root@host ~]$ yum install tuned  
[root@host ~]$ systemctl enable --now tuned
```

➤ **SELECTING A TUNING PROFILE**

The Tuned application provides profiles divided into the following categories:

- Power-saving profiles.
- Performance-boosting profiles.

The performance-boosting profiles include profiles that focus on the following aspects:

- Low latency for storage and network.
- High throughput for storage and network.
- Virtual machine performance.
- Virtualization host performance.

Tuning profiles distributed with red hat enterprise linux 9

TUNED PROFILE	PURPOSE
balanced	Ideal for systems that require a compromise between power saving and performance.
desktop	Derived from the balanced profile. Provides faster response of interactive applications.
throughput - performance	Tunes the system for maximum throughput.
latency - performance	Ideal for server systems that require low latency at the expense of power consumption.
network - latency	Derived from the latency - performance profile. It enables additional network tuning parameters to provide low network latency.
network - throughput	Derived from the throughput - performance profile. Additional network tuning parameters are applied for maximum network throughput.
powersave	Tunes the system for maximum power saving.
virtual-guest	Tunes the system for maximum performance if it runs on a virtual machine.
virtual-host	Tunes the system for maximum performance if it acts as a host for virtual machines.

Managing profiles from the command line

- The **tuned-adm** command is used to change settings of the **tuned** daemon. The **tuned-adm** command can query current settings, list available profiles, recommend a tuning profile for the system, change profiles directly, or turn off tuning.
- A system administrator identifies the currently active tuning profile with **tuned-adm active**.

```
[root@host ~]# tuned-adm active  
Current active profile: virtual-guest
```

- The **tuned-adm list** command lists all available tuning profiles, including both built-in profiles and custom tuning profiles created by a system administrator.

```
[root@host ~]# tuned-adm list
```

- Available profiles: ▪ Balanced ▪ Desktop ▪ Latency-performance.
- **Current active profile: Virtual-guest**

Managing profiles from the command line

- Use **tuned-adm profile** profilename to switch the active profile to a different one that better matches the system's current tuning requirements.

```
[root@host ~]$ tuned-adm profile throughput-performance
[root@host ~]$ tuned-adm active
Current active profile: throughput-performance
```

- The **tuned-adm** command can recommend a tuning profile for the system. This mechanism is used to determine the default profile of a system after installation.

```
[root@host ~]$ tuned-adm recommend
virtual-guest
```

- To revert the setting changes made by the current profile, either switch to another profile or deactivate the tuned daemon. Turn off tuned tuning activity with **tuned-adm off**.

```
[root@host ~]$ tuned-adm off
[root@host ~]$ tuned-adm active
No current active profile.
```


Influencing process scheduling



Change The Priority
Of A Process In
Linux

Linux process scheduling and multitasking:

- The way Linux (and other operating systems) can run more processes is by employing a technique called time-slicing.
- The part of the Linux kernel that performs this switching is called the process scheduler.
- There are exactly 40 different levels of niceness a process can have (-20 to 19).
- By default, processes will inherit their nice level from their parent, which is usually 0.
- Higher nice levels indicate less priority, while lower nice levels indicate a higher priority.
- Only root is allowed to set negative nice levels and lower the nice level on existing processes.
- Un privileged users are only allowed to set positive nice levels, and they are only allowed to raise the nice level on their existing process but cannot lower them.

How to set linux process priority using nice and renice commands

- Processes are scheduled according to priority.
- negative values are allowed only to root.

```
[root@lab01 ~]# ps l (To show nice values)
Or)
[root@lab01 ~]# ps axo user,pid,nice,command
[root@lab01 ~]# ps axo user,pid,command,nice --sort=nice
[root@lab01 ~]# ps axo user,pid,command,nice --sort=user
```

- The nice command is used to start a process with a user defined priority.

```
[root@lab01 ~]# nice vim text & (Default is 10)
[1] 9182
[root@lab01 ~]# nice -n vim text &
```

- The renice command is used to change the priority of a currently running process.

```
[root@lab01 ~]# renice 19 9182 (19 is the new value)
[root@lab01 ~]#top (press r to renice a process)
```

Summary

➤ In this chapter, you learned:

- The **tuned** service automatically modifies device settings to meet specific system needs based on a pre-defined selected tuning profile.
- To revert all changes made to system settings by a selected profile, either switch to another profile or deactivate the **tuned** service.
- The system assigns a relative priority to a process to determine its CPU access. This priority is called the **nice** value of a process.
- The **nice** command assigns a priority to a process when it starts. The **renice** command modifies the priority of a running process.



RH134

Controlling Access to Files with ACLs



Red Hat | Enterprise Linux

Goal

- Interpret and set Access Control Lists (ACLs) on files to handle situations requiring complex user and group access permissions.

Objectives

- Describe use cases for ACLs, identify files that have ACLs set, and interpret the effect of those ACLs.
- Set and remove ACLs on files and define default ACLs automatically set by a directory on newly created files.

Access control list

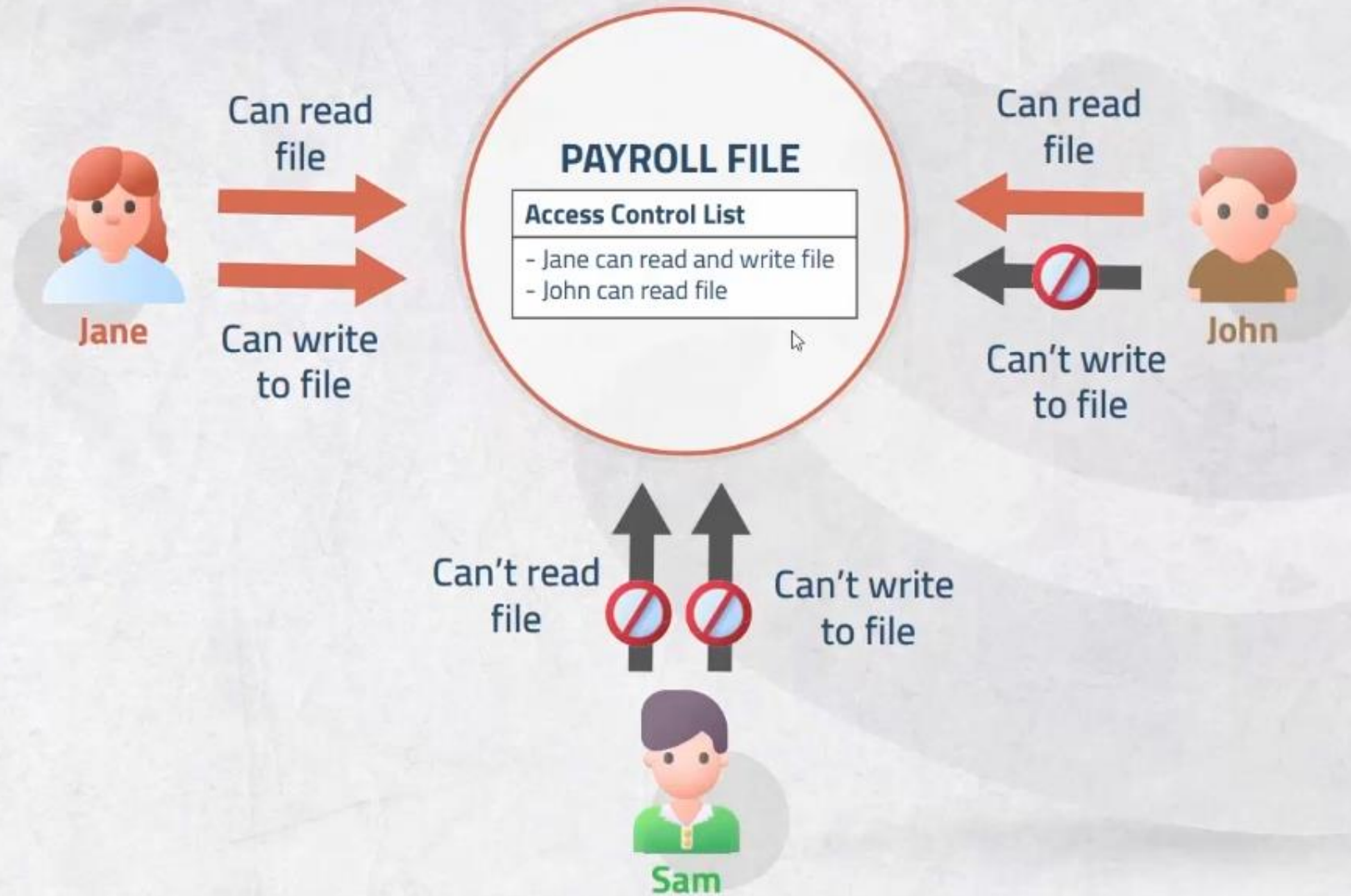
- ACL provides an additional flexible permission mechanism for file system on a Linux system. It enhance the traditional UNIX file permissions for files & folder. With ACL, you can give permissions for any user or any group with fine-grained access rights.
- ACLs can be configured per user, per group or via the effective rights mask. These permissions then can be applying to an individual user or a group, and you can use the same as rwx (Read, Write, Execute) found in regular file/ folder permissions.

Check kernel for ACL support

- To check ACL Support for file system and **POSIX_ACL=Y** option (if there is **N** instead of **Y**, then it means Kernel doesn't support ACL and need to be recompiled).

```
[root@lab01 ~]# grep -i acl /boot/config*  
CONFIG_EXT4_FS_POSIX_ACL=y  
CONFIG_XFS_POSIX_ACL=y  
CONFIG_BTRFS_FS_POSIX_ACL=y
```

Controlling Access to Files with (ACLs)



Access control list

- ACL provides an additional flexible permission mechanism for file system on a Linux system. It enhance the traditional UNIX file permissions for files & folder. With ACL, you can give permissions for any user or any group with fine-grained access rights.
- ACLs can be configured per user, per group or via the effective rights mask. These permissions then can be applying to an individual user or a group, and you can use the same as rwx (Read, Write, Execute) found in regular file/ folder permissions.

Check kernel for ACL support

- To check ACL Support for file system and **POSIX_ACL=Y** option (if there is **N** instead of **Y**, then it means Kernel doesn't support ACL and need to be recompiled).

```
[root@lab01 ~]# grep -i acl /boot/config*  
CONFIG_EXT4_FS_POSIX_ACL=y  
CONFIG_XFS_POSIX_ACL=y  
CONFIG_BTRFS_FS_POSIX_ACL=y
```

Viewing and interpreting ACL permissions

- The **ls -l** command only outputs minimal ACL setting details:

```
[user@host content]$ ls -l reports.txt  
-rwxrw----+ 1 user operators 130 Mar 19 23:56 reports.txt
```

- The plus sign (+) at the end of the 10-character permission string indicates that an extended ACL structure with entries exists on this file.

View File ACLs

- To display ACL settings on a file, use **getfacl** file:

```
[user@host content]$ getfacl reports.txt
```


View File ACLs

➤ To display ACL settings on a file, use getfacl file:

```
[user@host content]$ getfacl reports.txt
```

```
[user@host content]$ getfacl reports.txt  
# file: reports.txt  
# owner: user  
# group: operators  
user::rwx  
user:consultant3:---  
user:1005:rwx      #effective:rw-  
group::rwx        #effective:rw-  
group:consultant1:r--  
group:2210:rwx     #effective:rw-  
mask::rw-  
other::---
```

■ Commented entries:

- The first three lines are comments that identify the file name, owner (**user**), and group owner (**operators**).

If there are any additional file flags, such as **setuid** or **setgid**, then a fourth comment line will appear showing which flags are set.

```
# file: reports.txt  
# owner: user  
# group: operators
```

■ User entries:

- 1- File owner permissions. **user** has **rwX**.
- 2- Named user permissions. One entry for each named user associated with this file. **consultant3** has no permissions.
- 3- Named user permissions. UID **1005** has **rwX**, but the mask limits the effective permissions to **rw** only.

```
user::rwX 1  
user:consultant3:--- 2  
user:1005:rwX #effective:rw- 3
```


■ Commented entries:

- The first three lines are comments that identify the file name, owner (**user**), and group owner (**operators**).

If there are any additional file flags, such as **setuid** or **setgid**, then a fourth comment line will appear showing which flags are set.

```
# file: reports.txt  
# owner: user  
# group: operators
```

■ User entries:

- 1- File owner permissions. **user** has **rwX**.
- 2- Named user permissions. One entry for each named user associated with this file. **consultant3** has no permissions.
- 3- Named user permissions. UID **1005** has **rwX**, but the mask limits the effective permissions to **rw** only.

```
user::rwX 1  
user:consultant3:--- 2  
user:1005:rwX #effective:rw- 3
```

■ Group entries:

- 1- Group owner permissions. operators has **rw**x, but the mask limits the effective permissions to **rw** only.
- 2- Named group permissions. One entry for each named group associated with this file. consultant1 has **r** only.
- 3- Named group permissions. GID 2210 has **rw**x, but the mask limits the effective permissions to **rw** only.

3

```
group::rwx          #effective:rw-  1
group:consultant1:r--  2
group:2210:rwx       #effective:rw-  3
```


■ Mask entry:

- Mask settings show the maximum permissions possible for all named users, the group owner, and named groups. UID 1005, operators, and GID 2210 cannot execute this file, even though each entry has the execute permission set.

```
mask::rw-
```

■ Other entry:

- Other or "world" permissions. All other UIDs and GIDs have NO permissions.

```
other::---
```

View File ACLs

➤ To display ACL settings on a file, use getfacl file:

```
[user@host content]$ getfacl reports.txt
```

```
[user@host content]$ getfacl reports.txt
# file: reports.txt
# owner: user
# group: operators
user::rwx
user:consultant3:---
user:1005:rwx      #effective:rw-
group::rwx        #effective:rw-
group:consultant1:r--
group:2210:rwx     #effective:rw-
mask::rw-
other::---
```


The ACL mask

- The ACL mask defines the maximum permissions that you can grant to named users, the group owner, and named groups. It does not restrict the permissions of the file owner or other users. All files and directories that implement ACLs will have an ACL mask.
- The mask can be viewed with **getfacl** and explicitly set with **setfacl**. It will be calculated and added automatically if it is not explicitly set, but it could also be inherited from a parent directory default mask setting. By default, the mask is recalculated whenever any of the affected ACLs are added, modified, or deleted.

ACL permission precedence

When determining whether a process (a running program) can access a file, file permissions and ACLs are applied as follows:

- If the process is running as the user that owns the file, then the file's user ACL permissions apply.
- If the process is running as a user that is listed in a named user ACL entry, then the named user ACL permissions apply (as long as it is permitted by the mask).
- If the process is running as a group that matches the group owner of the file, or as a group with an explicitly named group ACL entry, then the matching ACL permissions apply (as long as it is permitted by the mask).
- Otherwise, the file's other ACL permissions apply.

How to set new ACLs

- Use the '**setfacl**' command for setting or modifying on any file or directory.

```
[root@lab01 ~]# setfacl -m u:karim:rw file.txt (We can use UID or username)
[root@lab01 ~]# setfacl -m o::rw file.txt
[root@lab01 ~]# setfacl -m g:sales:rw dir1
[root@lab01 ~]# setfacl -R -m g:sales:rw dir1 (We can also use GID or group name)
```

- You can add multiple entries with the same command; use a comma-separated list of entries:

```
[root@lab01 ~]# setfacl -m u::rwx,g:consultants:rx,o::- file.txt
```

- You can use the output from getfacl as input to setfacl, In this case, file-B will have the same ACL settings as file-A

```
[user@host ~]$ getfacl file-A | setfacl --set-file=- file-B
```

How to remove ACLs

- For removing ACL from any file/ directory, we use **x** and **b** options as shown below.

```
[root@lab01 ~]# setfacl -x u:karim file.txt      (To remove ACL permissions for the user karim)
[root@lab01 ~]# setfacl -b file.txt              (roll back to the default ACL)
```

■ -n, --no-mask

- ❑ Do not recalculate the effective rights mask. The default behavior of setfacl is to recalculate the ACL mask entry, unless a mask entry was explicitly given. The mask entry is set to the union of all permissions of the owning group, and all named user and group entries. (These are exactly the entries affected by the mask entry).

```
[root@server home]# setfacl -n -m u:karim:rw,passwd
[root@server home]# setfacl -m u:karim:rw,m::r passwd
```


How to remove ACLs

- For removing ACL from any file/ directory, we use **x** and **b** options as shown below.

```
[root@lab01 ~]# setfacl -x u:karim file.txt      (To remove ACL permissions for the user karim)
[root@lab01 ~]# setfacl -b file.txt              (roll back to the default ACL)
```

■ -n, --no-mask

- ❑ Do not recalculate the effective rights mask. The default behavior of setfacl is to recalculate the ACL mask entry, unless a mask entry was explicitly given. The mask entry is set to the union of all permissions of the owning group, and all named user and group entries. (These are exactly the entries affected by the mask entry).

```
[root@server home]# setfacl -n -m u:karim:rw,passwd
[root@server home]# setfacl -m u:karim:rw,m::r passwd
```

Summary

In this chapter, you learned:

- ACLs provide fine-grained access control to files and directories.
- The **getfacl** command displays the ACLs on a file or directory.
- The **setfacl** command sets, modifies, and removes default and standard ACLs on files and directories.



RH134

Managing SELinux Security



Red Hat | Enterprise Linux

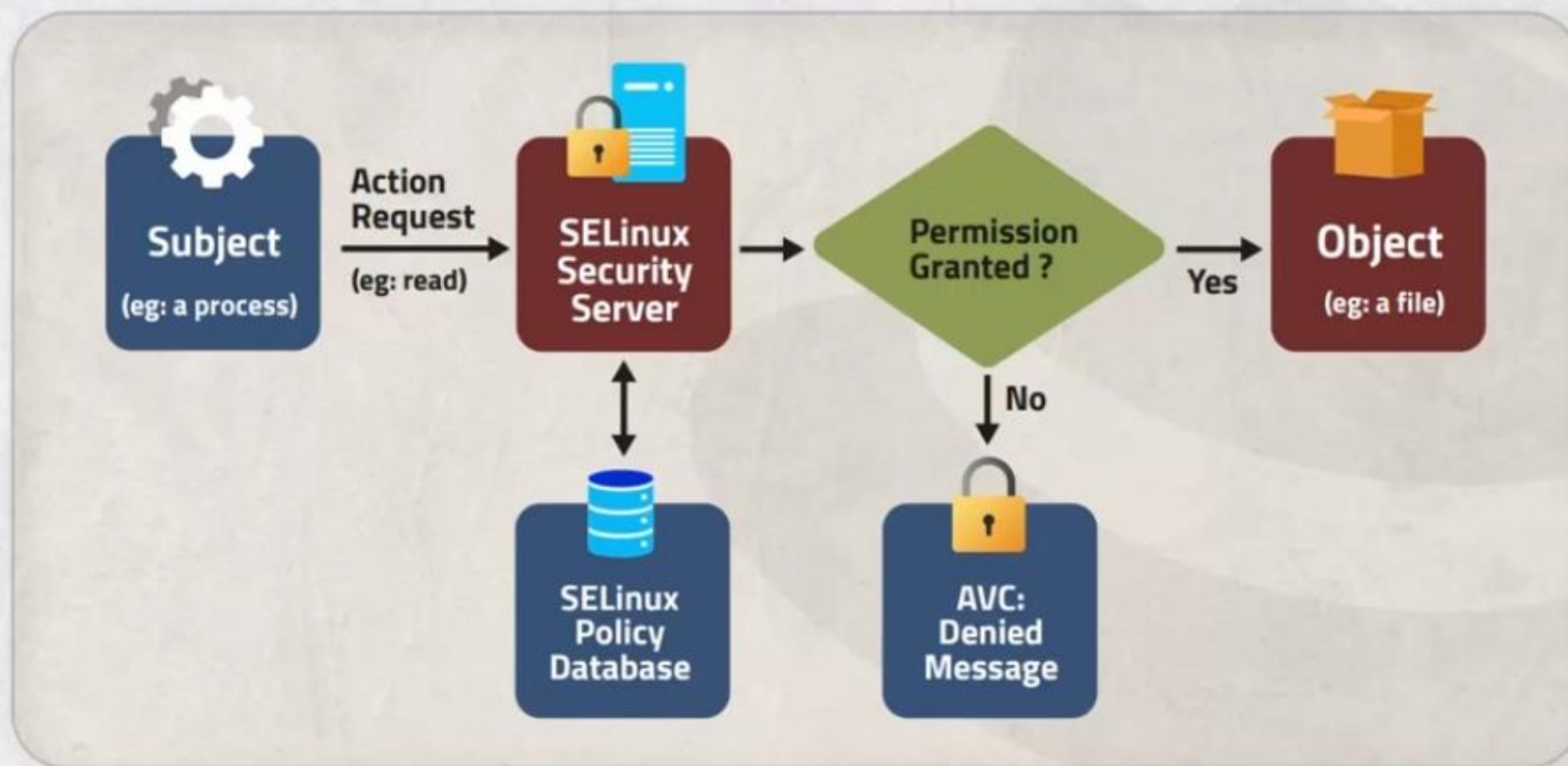
Goal

- Protect and manage the security of a server by using SELinux.

Objectives

- Describe how SELinux protects resources and how to select the enforcement mode.
- Configure a file's SELinux context to control how processes interact with that file.
- Configure SELinux booleans to allow runtime policy changes for varying access needs.
- Investigate SELinux log messages and troubleshoot SELinux AVC denials.

Managing SELinux Security



Why Use Security Enhanced Linux?

- Security Enhanced Linux (SELinux) is an **additional layer of system security**. The primary goal of SELinux is to **protect user data from system services that have been compromised**. Most Linux administrators are familiar with the standard user/group/other permission security model. This is a user and group based model known as discretionary access control. SELinux provides an additional layer of security that is **object-based** and controlled by more sophisticated rules, known as mandatory access control.
- To allow remote anonymous access to a web server, firewall ports must be opened. However, this gives malicious people an opportunity to crack the system through a security exploit. If they succeed in compromising the web server process they gain its permissions. Specifically, the permissions of the apache user and the apache group. That user and group has read access to the document root, **/var/www/html**. It also has access to **/tmp**, and **/var/tmp**, and any other files and directories that are world writable.

Why Use Security Enhanced Linux?

- SELinux is a set of security rules that determine which process can access which files, directories, and ports. Every file, process, directory, and port has a special security label called an **SELinux context**. **A context is a name used by the SELinux policy to determine whether a process can access a file, directory, or port.** By default, the policy does not allow any interaction unless an explicit rule grants access. If there is no allow rule, no access is allowed.
- SELinux labels have several contexts: **user**, **role**, **type**, and **sensitivity**. The targeted policy, which is the default policy enabled in Red Hat Enterprise Linux, bases its rules on the third context: the type context. Type context names usually end with **_t**.

unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/file2

SELinux User

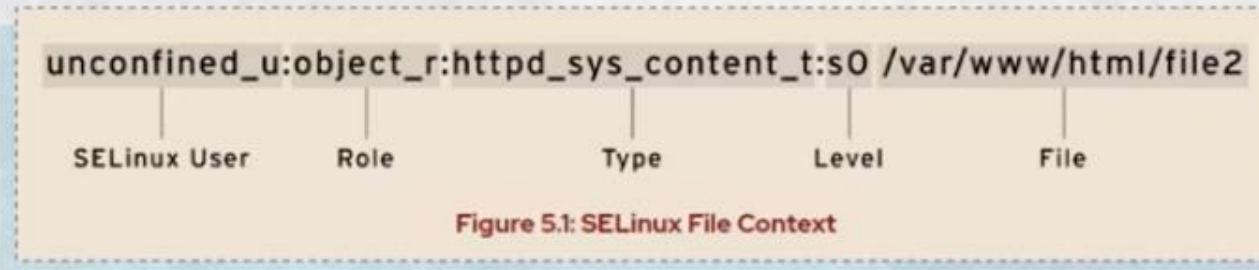
Role

Type

Level

File

Figure 5.1: SELinux File Context



- The type context for a web server is `httpd_t`. The type context for files and directories normally found in `/var/www/html` is `httpd_sys_content_t`. The contexts for files and directories normally found in `/tmp` and `/var/tmp` is `tmp_t`. The type context for web server ports is `http_port_t`.
- Apache has a type context of `httpd_t`. There is a policy rule that permits Apache access to files and directories with the `httpd_sys_content_t` type context. By default files found in `/var/www/html` and other web server directories have the `httpd_sys_content_t` type context. There is no `allow` rule in the policy for files normally found in `/tmp` and `/var/tmp`, so access is not permitted. With SELinux enabled, a malicious user who had compromised the web server process could not access the `/tmp` directory.

- The MariaDB server has a type context of **mysqld_t**. By default, files found in **/data/mysql** have the **mysqld_db_t** type context. This type context allows MariaDB access to those files but disables access by other services, such as the Apache web service.

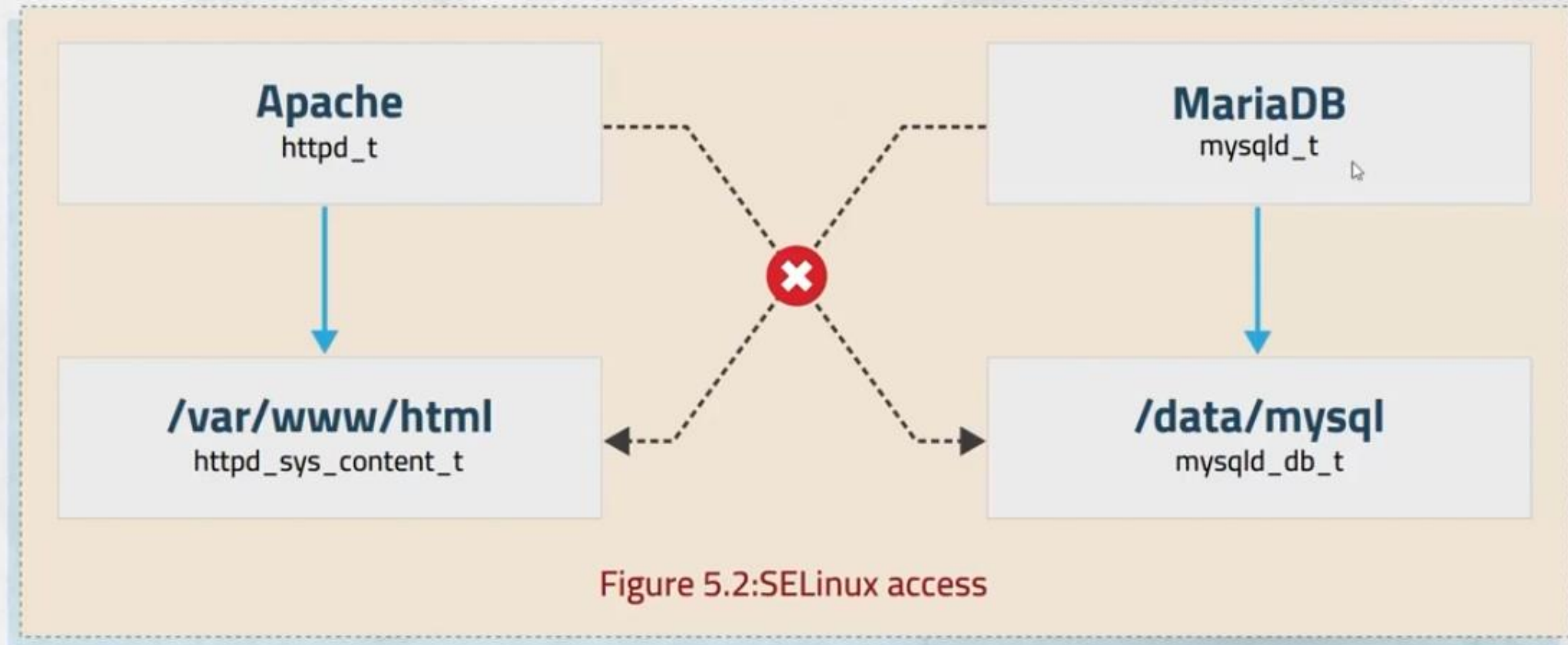


Figure 5.2:SELinux access

- Many commands that deal with files use the **-Z** option to display or set SELinux contexts. For instance, **ps**, **ls**, **cp**, and **mkdir** all use the **-Z** option to display or set SELinux contexts.

```
[root@host ~]# ps axZ
LABEL                PID TTY   STAT  TIME  COMMAND
system_u:system_r:init_t:s0 1?   Ss    0:09  /usr/lib/systemd/...
system_u:system_r:kernel_t:s0 2?   S     0:00  [kthreadd]
system_u:system_r:kernel_t:s0 3?   S     0:00  [ksoftirqd/0]
...output omitted...

[root@host ~]# systemctl start httpd
[root@host ~]# ps -ZC httpd
LABEL                PID TTY   TIME CMD
system_u:system_r:httpd_t:s0 1608? 00:00:05 httpd
system_u:system_r:httpd_t:s0 1609? 00:00:00 httpd
...output omitted...

[root@host ~]# ls -Z /home
drwx----- . root root system_u:object_r:lost_found_t:s0 lost+found
drwx----- . student student unconfined_u:object_r:user_home_dir_t:s0 student
drwx----- . visitor visitor unconfined_u:object_r:user_home_dir_t:s0 visitor

[root@host ~]# ls -Z /var/www
drwxr-xr-x. root root system_u:object_r:httpd_sys_script_exec_t:s0 cgi-bin
drwxr-xr-x. root root system_u:object_r:httpd_sys_content_t:s0 error
drwxr-xr-x. root root system_u:object_r:httpd_sys_content_t:s0 html
drwxr-xr-x. root root system_u:object_r:httpd_sys_content_t:s0 icons
```


SELinux Modes

- **Enforcing mode:** In enforcing mode, SELinux enforces its policies and denies access to any actions or processes that violate those policies. When a violation is detected.
- **Permissive mode:** In permissive mode, SELinux logs any violations but does not enforce its policies. This mode is useful for testing and troubleshooting SELinux policies without affecting system security.
- **Disabled mode:** In disabled mode, SELinux is completely turned off and has no effect on the system.

▪ Changing the current SELinux mode

- ❑ The SELinux subsystem provides tools to display and change modes. To determine the current SELinux mode, run the **getenforce** command. To set SELinux to a different mode, use the **setenforce** command:

Initial SELinux Context

- On systems running SELinux, all processes and files are labeled. The label represents the security relevant information, known as the SELinux context.
- New files typically inherit their SELinux context from the parent directory, thus ensuring that they have the proper context.
- But this inheritance procedure can be undermined in two different ways. First, if you create a file in a different location from the ultimate intended location and then move the file, the file still has the SELinux context of the directory where it was created, not the destination directory. Second, if you copy a file preserving the SELinux context, as with the `cp -a` command, the SELinux context reflects the location of the original file.
- The following example demonstrates inheritance and its pitfalls. Consider these two files created in `/tmp`, one moved to `/var/www/html` and the second one copied to the same directory. Note the SELinux contexts on the files. The file that was moved to the `/var/www/html` directory retains the file context for the `/tmp` directory. The file that was copied to the `/var/www/html` directory inherited SELinux context from the `/var/www/html` directory.

Changing The SELinux Context of a File

- Commands to change the SELinux context on files include **semanage fcontext**, **restorecon**, and **chcon**.
- The preferred method to set the SELinux context for a file is to declare the default labeling for a file using the **semanage fcontext** command and then applying that context to the file using the **restorecon** command. This ensures that the labeling will be as desired even after a complete relabeling of the file system.
- The **chcon** command changes SELinux contexts. **chcon** sets the security context on the file, stored in the file system. It is useful for testing and experimenting. However, it does not save context changes in the SELinux context database. When a **restorecon** command runs, changes made by the **chcon** command also do not survive. Also, if the entire file system is relabeled, the SELinux context for files changed using **chcon** are reverted.

Defining SELinux Default File Context Rules

- The following table is a reference for **semanage fcontext** options to add, remove or list SELinux file contexts.

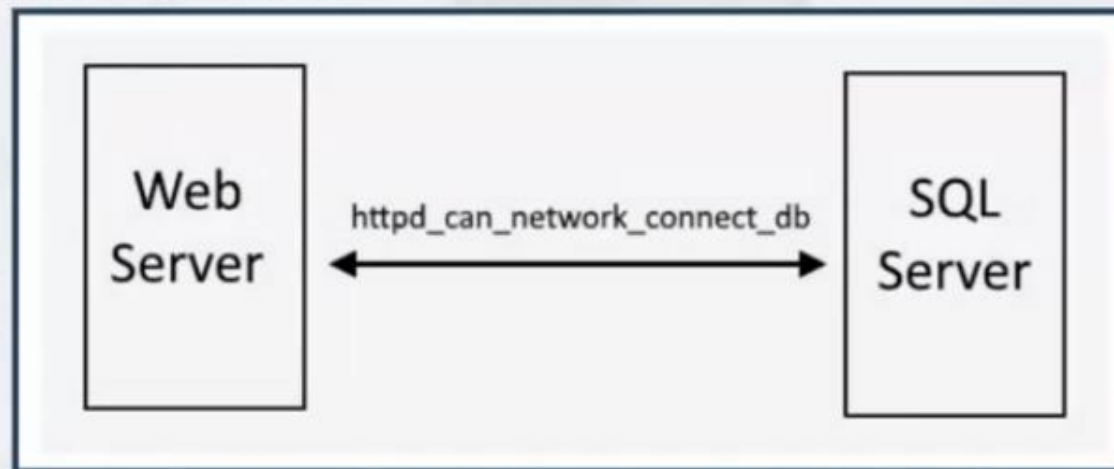
OPTION	DESCRIPTION
-a, --add	Add a record of the specified object type
-d, --delete	Delete a record of the specified object type
-l, --list	List records of the specified object type

- To ensure that all files in a directory have the correct file context run the **semanage fcontext -l** followed by the **restorecon** command.

```
[root@host ~]# semanage fcontext -l  
[root@host; ~]# restorecon -Rv /var/www/
```


SELinux Booleans

- SELinux booleans are switches that change the behavior of the SELinux policy. SELinux Booleans are rules that can be enabled or disabled. They can be used by security administrators to tune the policy to make selective adjustments.
- For example, let's say you have a Linux web server that will be using a SQL database, such as MySQL or MariaDB. One of the SELinux booleans is **httpd_can_network_connect_db**. If `httpd_can_network_connect_db` is off, the web server will be unable to connect to the SQL database. On the other hand, if `httpd_can_network_connect_db` is on, the web server will be able to connect to the SQL database.



- Commands useful for managing SELinux booleans include **getsebool**, which lists booleans and their state, and **setsebool** which modifies booleans. **setsebool -P** modifies the SELinux policy to make the modification persistent. And **semanage boolean -l** reports on whether or not a boolean is persistent, along with a short description of the boolean.
- Non-privileged users can run the **getsebool** command, but you must be a superuser to run **semanage boolean -l** and **setsebool -P**.

```
[user@host ~]$ getsebool -a
[user@host ~]$ getsebool httpd_can_network_connect_db
httpd_can_network_connect_db --> off

[user@host ~]$ setsebool httpd_can_network_connect_db on
[user@host ~]$ semanage boolean -l | grep httpd_can_network_connect_db
httpd_can_network_connect_db (on , off) Allow httpd to can network connect db
```


Troubleshooting SELinux Issues

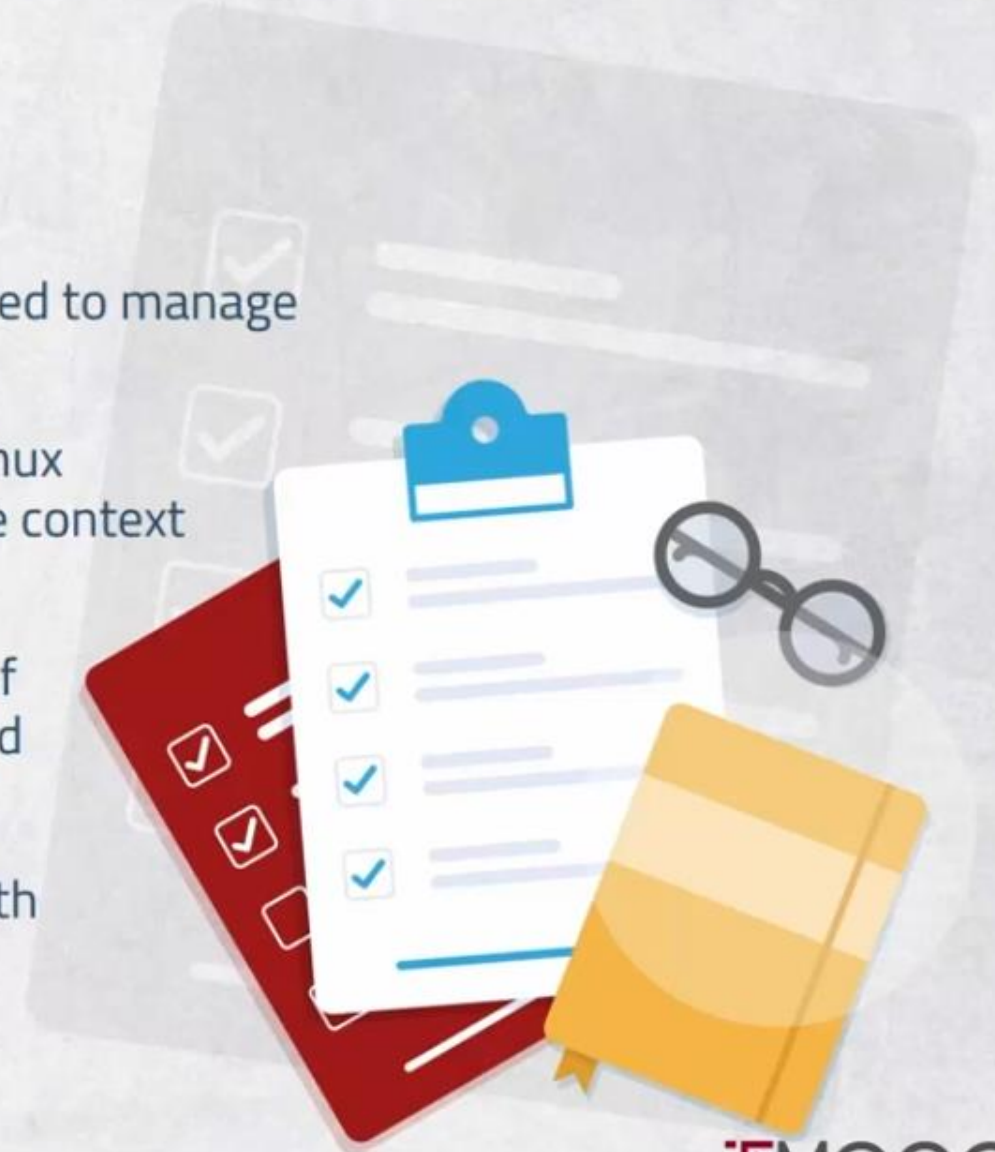
- It is important to understand what actions you must take when SELinux prevents access to files on a server that you know should be accessible.
- Install the `setroubleshoot-server` package to send SELinux messages to `/var/log/messages`. `setroubleshoot-server` listens for audit messages in `/var/log/audit/audit.log` and sends a short summary to `/var/log/messages`. This summary includes unique identifiers (*UUID*) for SELinux violations that can be used to gather further information. The `sealert -l UUID` command is used to produce a report for a specific incident.
- Use `sealert -a /var/log/audit/audit.log` to produce reports for all incidents in that file.

```
[root@host ~]# ausearch -m AVC -ts recent
```

Summary

In this chapter, you learned:

- The **getenforce** and **setenforce** commands are used to manage the SELinux mode of a system.
- The **semanage** command is used to manage SELinux policy rules. The **restorecon** command applies the context defined by the policy.
- Booleans are switches that change the behavior of the SELinux policy. They can be enabled or disabled and are used to tune the policy.
- The **sealert** displays useful information to help with SELinux troubleshooting.





RH134

Managing Basic Storage



Red Hat | Enterprise Linux

Goal

- Create and manage storage devices, partitions, file systems, and swap spaces from the command line.

Objectives

- Create storage partitions, format them with file systems, and mount them for use.
- Create and manage swap spaces to supplement physical memory.

File Systems

What is a file system in Linux or Unix?

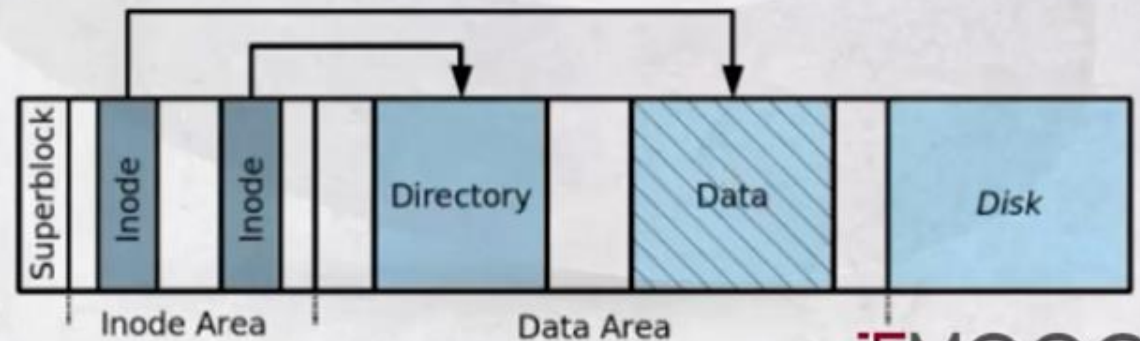
- File system is a collection of files and directories stored on disk. Each file system is stored in a separate whole disk partition. The following are a few of the file system:

But what is in a File system?

- File system divided into two categories:
 - **User data** – Stores actual data contained in files.
 - **Metadata** – Stores file system structural information such as superblock, inodes, directories.

Partition divided into blocks

- Super block
- Inode blocks
- Data block



What is Superblock?

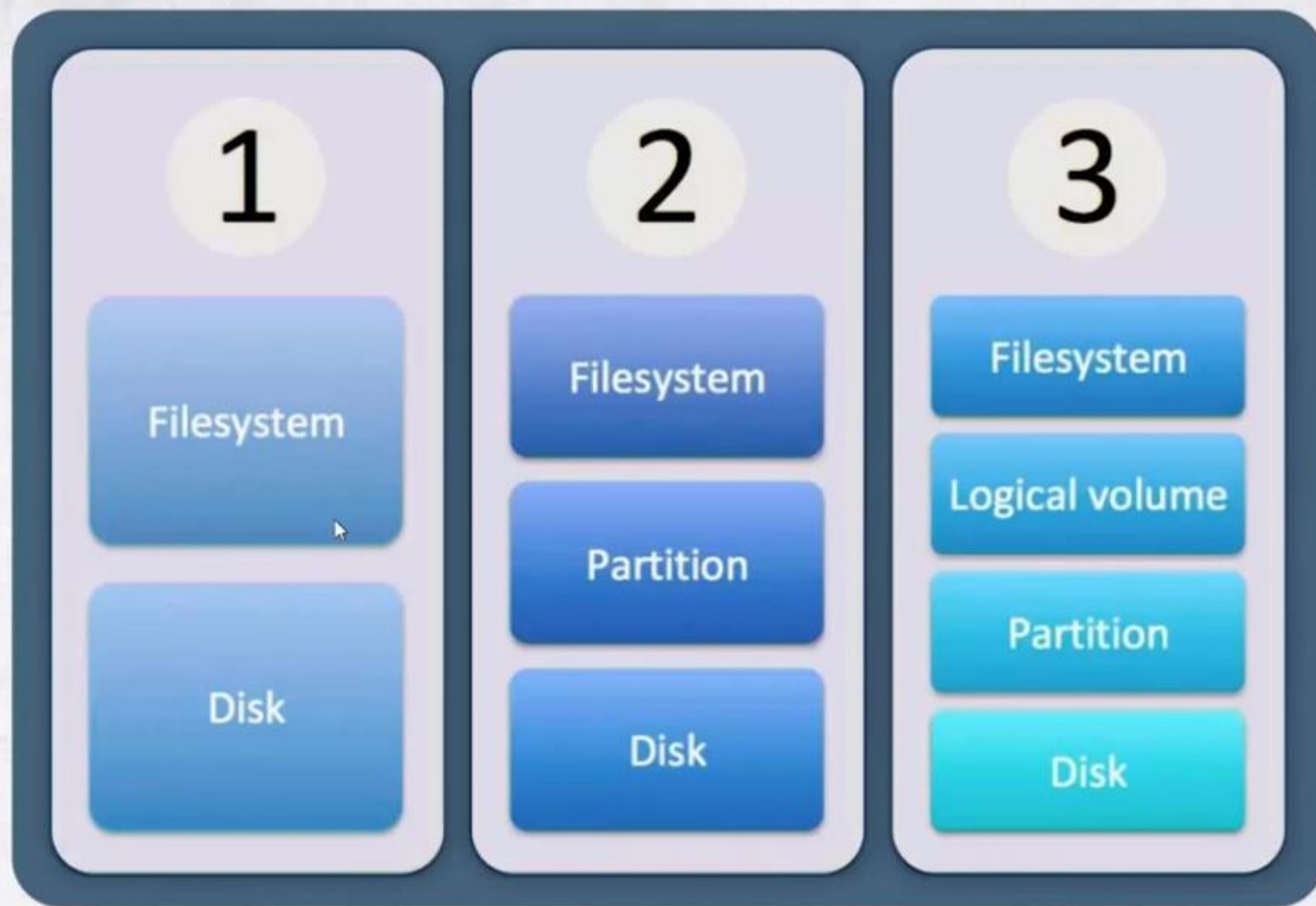
- Each file system is different, and they have type like ext2, ext3 etc.
- Further each file system has size like 5 GB, 10 GB and status such as mount status. In short, each file system has a superblock, which contains information about file system such as:
 - File system type
 - Status
 - Size
 - Information about other metadata structures

What is Inodes?

➤ Metadata about the files

- File type (executable, block special etc)
- Permissions (read, write etc)
- Owner
- Group
- File size
- File access, change and modification time
- Number of links (soft/hard)
- Access Control List (ACLs)

Adding Disks Partitions and File Systems to a Linux System

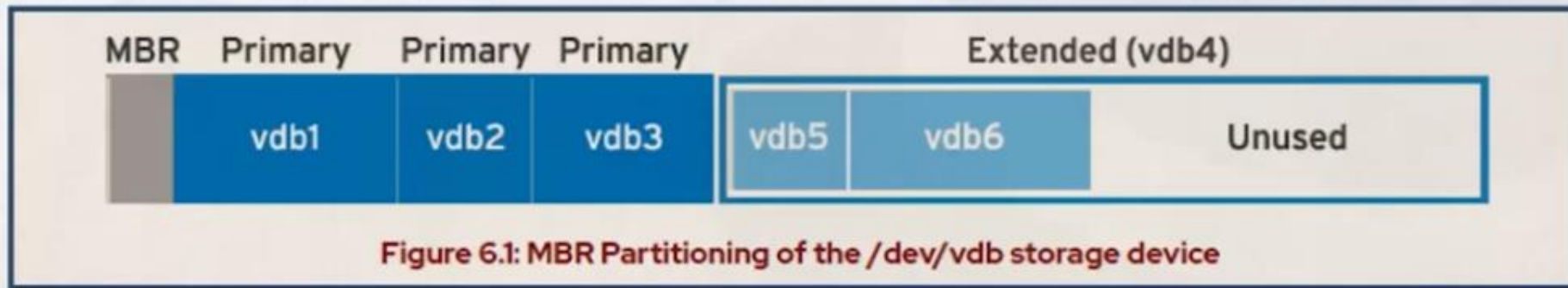


Partitioning a Disk

- Disk partitioning allows system administrators to divide a hard drive into multiple logical storage units, referred to as partitions. By separating a disk into partitions, system administrators can use different partitions to perform different functions.
- **For example, disk partitioning is necessary or beneficial in these situations:**
 - Limit available space to applications or users.
 - Separate operating system and program files from user files.
 - Create a separate area for memory swapping.
 - Limit disk space use to improve the performance of diagnostic tools and backup imaging.

MBR Partitioning Scheme

- Since 1982, the Master Boot Record (**MBR**) partitioning scheme has dictated how disks are partitioned on systems. This scheme supports a maximum of **four primary partitions**. On Linux systems, with the use of extended and logical partitions, administrators can create a maximum of **15 partitions**. Because partition size data is stored as **32-bit values**, disks partitioned with the MBR scheme have a maximum disk and partition size of **2 TiB**.

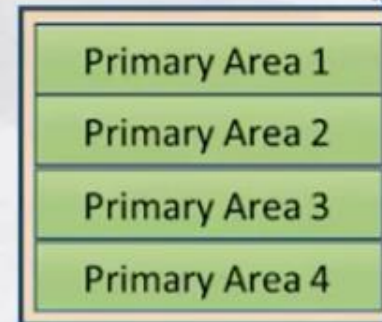


- The **2 TiB disk** and partition size limit of the MBR partitioning scheme is no longer a theoretical limit but rather a real-world problem that system administrators encounter more and more frequently in production environments. As a result, the legacy **MBR scheme** is in the process of being superseded by the new **GUID Partition Table (GPT)** for disk partitioning.

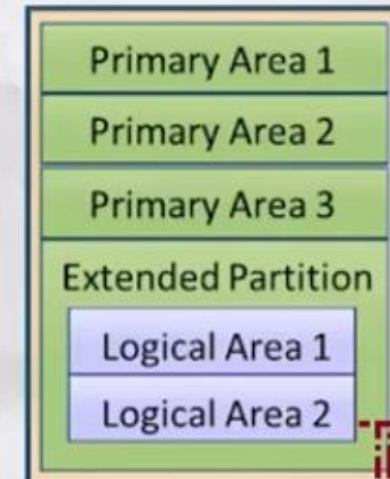
Primary, Extended, and Logical partition types

- **Primary partition** – These are the convention partitions that would create by default. If you only plan to create 4 partitions or less, then you only need to create primary partition and you don't have to worry about extended and logical partitions.
- **Extended partition** – If you want to create more than 4 partitions on a hdd, now or in the future, then you need to create one of the partition as an extended partition. An extended partition is not actually a partition, instead it is essentially an empty container inside which you can create (logical) partitions. You cannot store data inside an extended partition directly, instead you first have to create logical partition inside the extended partition. You can only create one extended partition per hdd.
- **Logical partition** – These are the names of any partitions created inside an extended partition, OS uses logical partitions in the same was as a typical primary partition.

Primary Area Only

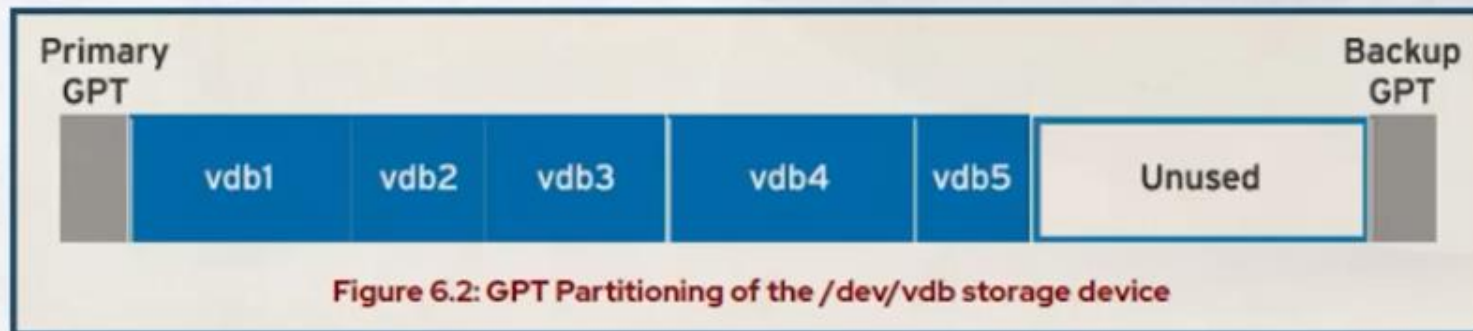


Extended Area Only



GPT Partitioning Scheme

- A GPT provides a maximum of **128 partitions**. Unlike an MBR, which uses 32 bits for storing logical block addresses and size information, a GPT allocates **64 bits** for logical block addresses.
- Allows a GPT to accommodate partitions and disks of up to eight zebibytes (ZiB) or eight billion tebibytes.
- In addition to addressing the limitations of the MBR partitioning scheme, a GPT also offers some additional features and benefits. A GPT uses a **globally unique identifier (GUID)** to identify each disk and partition. In contrast to an MBR, which has a single point of failure, a GPT offers redundancy of its partition table information. The primary GPT resides at the head of the disk, while a backup copy, the secondary GPT, is housed at the end of the disk. A GPT uses a checksum to detect errors and corruptions in the GPT header and partition table.



Should I Use MBR or GPT in Linux?

One of the biggest advantages of **GPT** over **MBR** is, on an MBR disk, the partitioning and boot data is stored in one place. If this data is corrupted, you're in trouble meanwhile in GPT stores multiple copies of this data across the disk, so you can recover if the data is corrupted.

Managing Partitions With **Parted**

- Partition editors are programs which allow administrators to make changes to a disk's partitions, such as creating partitions, deleting partitions, and changing partition types. To perform these operations, administrators can use the Parted partition editor for both the MBR and the GPT partitioning scheme.
- The **parted** command takes the device name of the whole disk as the first argument and one or more subcommands. The following example uses the **print** subcommand to display the partition table on the **/dev/vda** disk.

```
[root@host ~]# parted /dev/nvme0n1 print
```

- If you do not provide a subcommand, **parted** opens an interactive session for issuing commands.

```
[root@host ~]# parted /dev/nvme0n1
```

- By default, **parted** displays all the sizes in powers of 10 (KB, MB, GB). You can change that default with the **unit** subcommand which accepts the following parameters:

- **s** for sector
- **B** for byte
- **MB**, **GB**, or **TB** (powers of 10)

```
[root@host ~]# parted /dev/nvme0n1 unit s print
[root@host ~]# parted /dev/nvme0n1 unit B print
[root@host ~]# parted /dev/nvme0n1 unit MB print
[root@host ~]# parted /dev/nvme0n1 unit GB print
[root@host ~]# parted /dev/nvme0n1 unit TB print
```


Writing the Partition Table on a New Disk

- To partition a new drive, you first have to write a disk label to it. The disk label indicates which partitioning scheme to use.
- Keep in mind that **parted** makes the changes immediately. A mistake with **parted** could definitely lead to data loss.
- As the root user, **use the following command to write an MBR disk label to a disk:**

```
[root@host ~]# parted /dev/nvme0n1 mklabel msdos
```

- To write a GPT disk label, **use the following command:**

```
[root@host ~]# parted /dev/nvme0n1 mklabel gpt
```

- The **mklabel** subcommand wipes the existing partition table. Only use **mklabel** when the intent is to reuse the disk without regard to the existing data. If a new label changes the partition boundaries, all data in existing file systems will become inaccessible.

Creating MBR Partitions

- 1- Specify the disk device to create the partition on.

```
[root@host ~]# parted /dev/vdb
```

- As the root user, execute the **parted** command and specify the disk device name as an argument. This starts the **parted** command in interactive mode and displays a command prompt.
- 2- Use the **mkpart** subcommand to create a new primary or extended partition.

```
(parted) mkpart  
Partition type? primary/extended? primary
```

- For situations where you need more than four partitions on an MBR-partitioned disk, create three primary partitions and one extended partition. This extended partition serves as a container within which you can create multiple logical partitions.

Creating MBR Partitions

- 3- Indicate the file-system type that you want to create on the partition, such as xfs or ext4. This does not create the file system on the partition; it is only an indication of the partition type.

```
File system type? [ext2]? xfs
```

- ➡ To get the list of the supported file-system types, use the following command:

```
[root@host ~]# parted /dev/vdb help mkpart
```

- 4- Specify the sector on the disk that the new partition starts on.

```
Start? 2048s
```

Notice the **s** suffix to provide the value in sectors. You can also use the **MB**, **GB**, or **TB** suffixes. If you do not provide a suffix, **MB** is the default.

Creating MBR Partitions

5- Specify the disk sector where the new partition should end.

```
End? 1000MB
```

➤ As soon as you provide the end position, **parted** updates the partition table on the disk with the new partition details.

6- Exit **parted**.

```
(parted) quit
```

```
Information: You may need to update /etc/fstab.
```

7- Run the **udevadm settle** command. This command waits for the system to detect the new partition and to create the associated device file under the **/dev** directory. It only returns when it is done.

```
[root@host ~]# udevadm settle
```


Creating MBR Partitions

- As an alternative to the interactive mode, you can also create the partition as follows:

```
[root@host ~]# parted /dev/vdb mkpart primary xfs 2048s 1000MB
```

- As an alternative to the interactive mode, you can also create the partition as follows:

```
parted /dev/nvme0n2 <<END  
mkpart primary 0% 10%  
quit  
END
```

Creating GPT Partitions

- The GTP scheme also uses the **parted** command to create new partitions.

1- Specify the disk device to create the partition on.

```
[root@host ~]# parted /dev/vdb
```

2- Use the **mkpart** subcommand to start creating the new partition.

- With the GPT scheme, each partition is given a name.

```
(parted) mkpart  
Partition name? []? usersdata
```

3- Indicate the file system type that you want to create on the partition.

```
File system type? [ext2]? xfs
```


Creating GPT Partitions

4- Specify the sector on the disk that the new partition starts on.

Start? **2048s**

5- Specify the disk sector where the new partition should end.

End? **1000MB**

6- Exit **parted**.

(parted) **quit**

Information: You may need to update /etc/fstab.

7- Run the **udevadm settle** command.

[root@host ~]# **udevadm settle**

➤ As an alternative to the interactive mode, you can also create the partition as follows:

[root@host ~]# **parted /dev/vdb mkpart primary xfs 2048s 1000MB**

Deleting Partitions

➤ The following steps apply for both the MBR and GPT partitioning schemes.

1- Specify the disk that contains the partition to be removed.

```
[root@host ~]# parted /dev/vdb
```

2- Identify the partition number of the partition to delete.

```
(parted) print
```

3- Delete the partition.

```
(parted) rm1
```

➤ The **rm** subcommand immediately deletes the partition from the partition table on the disk.

4- Exit **parted**.

```
(parted) quit
```


Creating File Systems

- After the creation of a block device, the next step is to add a file system to it. Red Hat Enterprise Linux supports many different file system types, but two common ones are **XFS** and **ext4**, the installer for Red Hat Enterprise Linux, uses **XFS** by default.
- As root, use the **mkfs.xfs** command to apply an XFS file system to a block device. For ext4, use **mkfs.ext4**.

```
[root@host ~]# mkfs.xfs /dev/vdb1
```

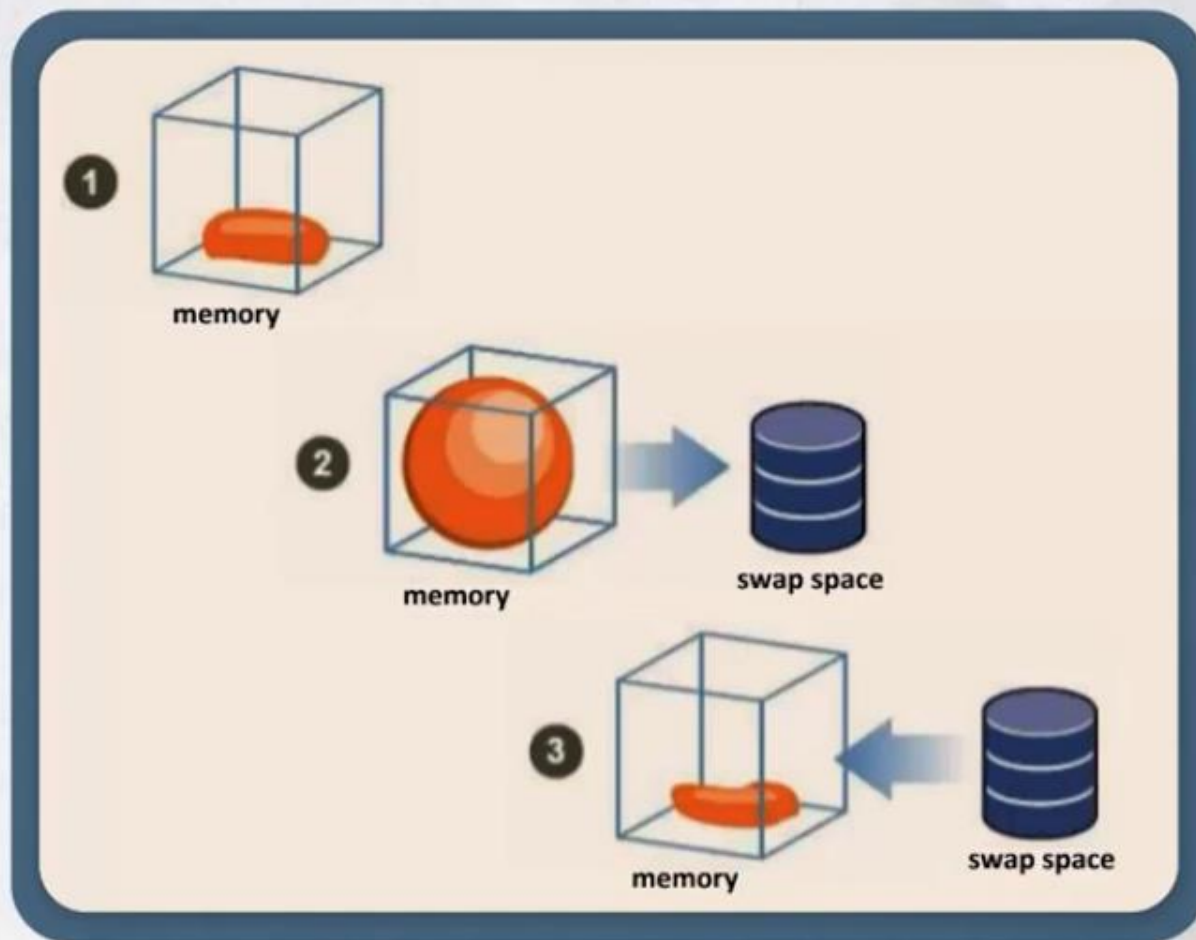
Mounting File Systems

- After you have added the file system, the last step is to mount the file system to a directory in the directory structure. When you mount a file system onto the directory hierarchy, user can access or write files on the device.

```
[root@host ~]# mount /dev/vdb1 /mnt  
[root@host ~]# cat /etc/fstab
```

- When you add or remove an entry in the **/etc/fstab** file, run the **systemctl daemon-reload** command, or reboot the server, for systemd to register the new configuration.

Managing swap space



Introducing SWAP Space Concepts

- A *swap space* is an area of a disk under the control of the Linux kernel memory management subsystem. The kernel uses swap space to supplement the system RAM by holding inactive pages of memory. The combined system RAM plus swap space is called *virtual memory*.
- When the memory usage on a system exceeds a defined limit, the kernel searches through RAM looking for idle memory pages assigned to processes. The kernel writes the idle pages to the swap area and reassigns the RAM pages to other processes. If a program requires access to a page on disk, the kernel locates another idle page of memory, writes it to disk, then recalls the needed page from the swap area.
- Because swap areas reside on disk, swap is slow when compared with RAM. While it is used to augment system RAM, you should not consider swap space as a sustainable solution for insufficient RAM for your workload.

Sizing the SWAP Space

- Administrators should size the swap space based on the memory workload on the system. Application vendors sometimes provide recommendations on that subject. **The following table provides some guidance based on the total amount of physical memory:**

RAM	SWAP SPACE
<i>2 GiB or less</i>	Twice the RAM
<i>Between 2 GiB and 8 GiB</i>	Same as RAM
<i>Between 8 GiB and 64 GiB</i>	At least 4 GiB
<i>More than 64 GiB</i>	At least 4 GiB

SWAP Partition

```
[root@host ~]# parted /dev/vdb
```

```
(parted) mkpart
```

```
Partition name? []? Swap1
```

```
File system type? [ext2]? linux-swap
```

```
Start? 1001MB
```

```
End? 1257MB
```

```
(parted) print
```

```
(parted) quit
```

```
[root@host ~]# udevadm settle
```

Formatting the Device

```
[root@host ~]# mkswap /dev/vdb2
```

Activating a SWAP Space

```
[root@host ~]# swapon /dev/vdb2
```

SWAP File

```
# dd if=/dev/random of=/swap_file bs=1M count=1024
```

```
# fallocate -l 1G /swap_file
```

```
# mkswap /swap_file (Format the device)
```

```
# swapon /swap_file (Activate a swap space)
```

```
# swapon -a (activate all swap spaces listed in the  
/etc/fstab file)
```

```
# vi /etc/fstab (Persistently activate swap space)
```

```
# systemctl daemon-reload
```

```
# free -m (show the swap file system)
```

```
# swapoff /swap_file (deactivate a swap space)
```


Setting the SWAP Space Priority

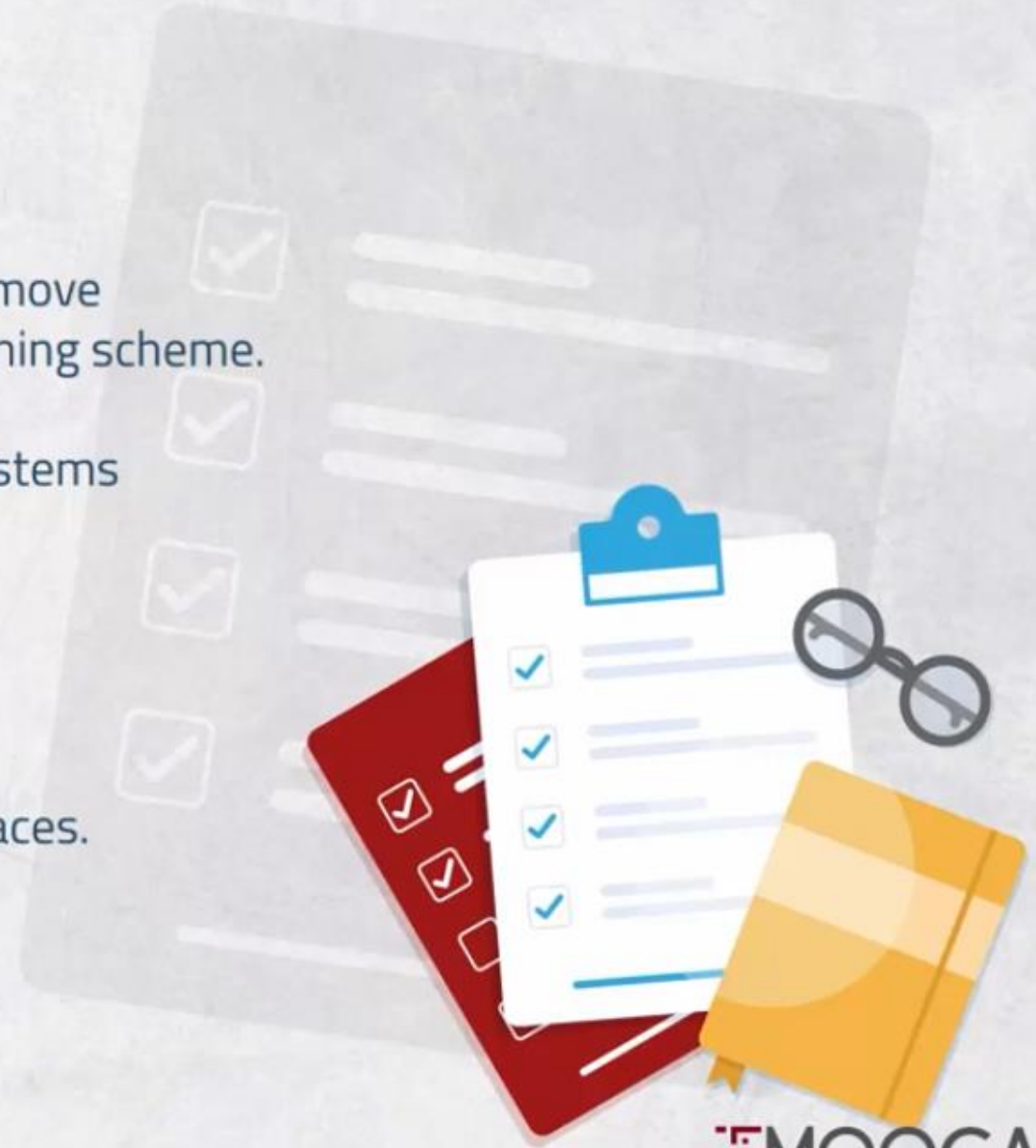
- By default, the system uses swap spaces in series, meaning that the kernel uses the first activated swap space until it is full, then it starts using the second swap space. However, you can define a priority for each swap space to force that order.
- To set the priority, use the `pri` option in `/etc/fstab`. The kernel uses the swap space with the highest priority first. The default priority is `-2`.
- The following example shows three swap spaces defined in `/etc/fstab`. The kernel uses the last entry first, with `pri=10`. When that space is full, it uses the second entry, with `pri=4`. Finally, it uses the first entry, which has a default priority of `-2`.
- Use `swapon --show` to display the swap space priorities.
- When swap spaces have the same priority, the kernel writes to them in a round-robin

UUID=af30cbb0-3866-466a-825a-58889a49ef33	swap	swap	defaults	0 0
UUID=39e2667a-9458-42fe-9665-c5c854605881	swap	swap	pri=4	0 0
UUID=fbd7fa60-b781-44a8-961b-37ac3ef572bf	swap	swap	pri=10	0 0

Summary

In this chapter, you learned:

- You use the **parted** command to add, modify, and remove partitions on disks with the MBR or the GPT partitioning scheme.
- You use the **mkfs.xfs** command to create XFS file systems on disk partitions.
- You need to add file-system mount commands to **/etc/fstab** to make those mounts persistent.
- You use the **mkswap** command to initialize swap spaces.



RH134

Managing Logical Volumes



Red Hat | Enterprise Linux

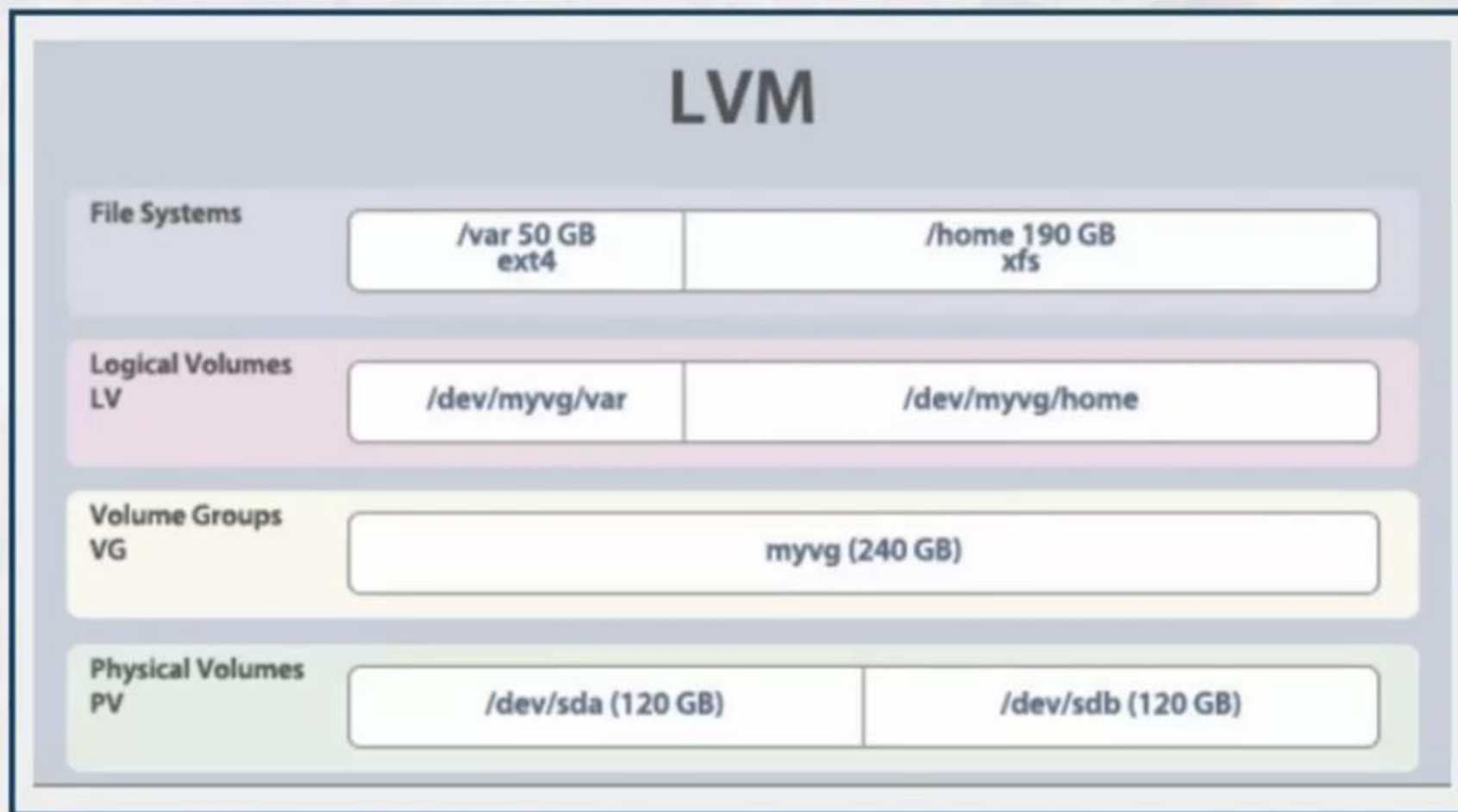
Goal

- Create and manage logical volumes containing file systems and swap spaces from the command line.

Objectives

- Create and manage logical volumes from storage devices, and format them with file systems or prepare them with swap spaces.
- Add and remove storage assigned to volume groups, and nondestructively extend the size of a logical volume formatted with a file system.

Managing Logical Volume Management (LVM) Storage



Logical Volume Managment (LVM) Concepts

Logical volumes and logical volume management make it easier to manage disk space. If a file system that hosts a logical volume needs more space, it can be allocated to its logical volume from the free space in its volume group and the file system can be resized. If a disk starts to fail, a replacement disk can be registered as a physical volume with the volume group and the logical volume's extents can be migrated to the new disk.

LVM Definitions

➤ Physical devices

Physical devices are the storage devices used to save data stored in a logical volume. These are block devices and could be disk partitions, whole disks, RAID arrays, or SAN disks. A device must be initialized as an LVM physical volume in order to be used with LVM. The entire device will be used as a physical volume.

LVM Definitions

➤ Physical volumes (PVs)

You must initialize a device as a physical volume before using it in an LVM system. LVM tools segment physical volumes into physical extents (PEs), which are small chunks of data that act as the smallest storage block on a physical volume.

➤ Volume groups (VGs)

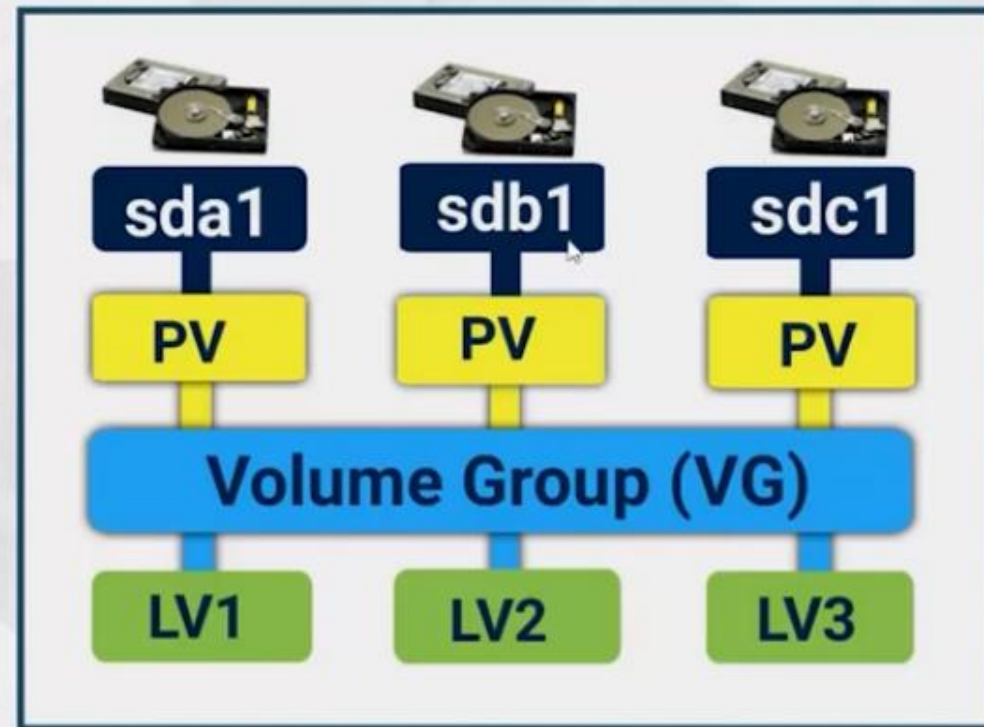
Volume groups are storage pools made up of one or more physical volumes. This is the functional equivalent of a whole disk in basic storage. A PV can only be allocated to a single VG. A VG can consist of unused space and any number of logical volumes.

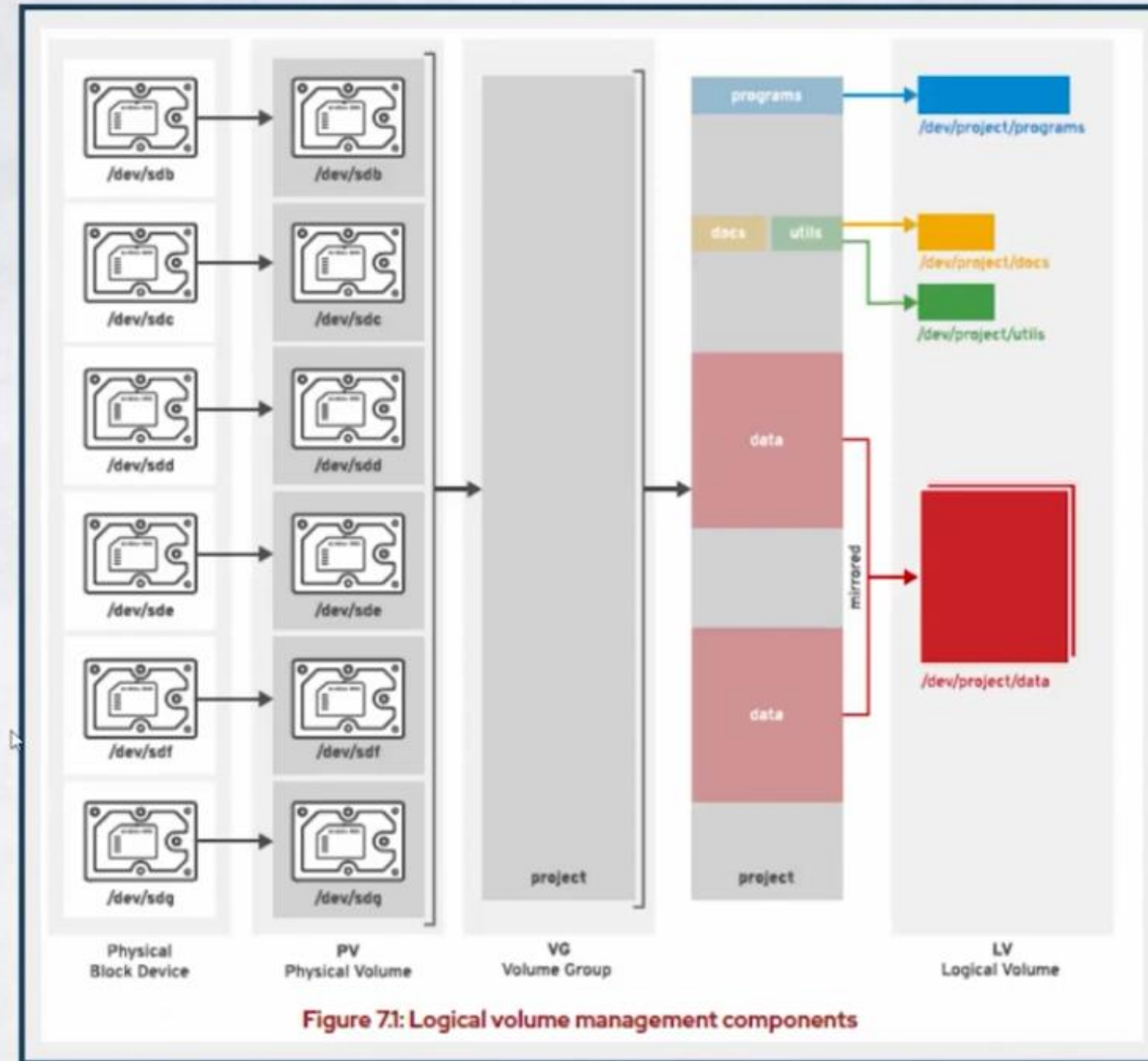
➤ Logical volumes (LVs)

Logical volumes are created from free physical extents in a volume group and provide the "storage" device used by applications, users, and the operating system. LVs are a collection of logical extents (LEs), which map to physical extents, the smallest storage chunk of a PV. By default, each LE maps to one PE. Setting specific LV options changes this mapping; for example, mirroring causes each LE to map to two PEs.

Why Logical volume management (LVM) ?

- Volumes can consist of more than one disk.
- Easy resize operation.
- Easy replacement of failing disks.
- Advanced options such a working with snapshots.
- Easy to add new volumes.
- Easy to add many volumes.
- Up to 256 logical volume.





Create a physical Volume

- Use **pvcreate** to label the partition (or other physical device) as a physical volume. The **pvcreate** command divides the physical volume into physical extents (PEs) of a fixed size, for example, 4 MiB blocks. You can label multiple devices at the same time by using space-delimited device names as arguments to **pvcreate**.

```
[root@host ~]# pvcreate /dev/vdb2 /dev/vdb1
```

Create a Volume Group

- Use **vgcreate** to collect one or more physical volumes into a volume group. A volume group is the functional equivalent of a hard disk; you will create logical volumes from the pool of free physical extents in the volume group.
- The **vgcreate** command-line consists of a volume group name followed by one or more physical volumes to allocate to this volume group.

```
[root@host ~]# vgcreate vg01 /dev/vdb2 /dev/vdb1
```

- This creates a VG called vg01 that is the combined size, in PE units, of the two PVs **/dev/vdb2** and **/dev/vdb1**.
- A VG only needs to be created if none already exist. Additional VGs may be created for administrative reasons to manage the use of PVs and LVs. Otherwise, existing VGs can be extended to accommodate new LVs when needed.

Creating a Logical Volume

- Use **lvcreate** to create a new logical volume from the available physical extents in a volume group. At a minimum, the **lvcreate** command includes the **-n** option to set the LV name, either the **-L** option to set the LV size in bytes or the **-l** option to set the LV size in extents, and the name of the volume group hosting this logical volume.

```
[root@host ~]# lvcreate -n lv01 -L 700M vg01
```

- This creates an LV called lv01, 700 MiB in size, in the VG vg01. This command will fail if the volume group does not have a sufficient number of free physical extents for the requested size.
- Note also that the size will be rounded to a factor of the physical extent size if the size cannot match exactly.
- The following list provides some examples of creating LVs:
 - **lvcreate -L 128M**: Size the logical volume to exactly 128 MiB.
 - **lvcreate -l 128**: Size the logical volume to exactly 128 extents.

Add the file system

- Use **mkfs** to create an **XFS** file system on the new logical volume. Alternatively, create a file system based on your preferred file system, for example, **ext4**.

```
[root@host ~]# mkfs -t xfs /dev/vg01/lv01
```

- To make the file system available across reboots, perform the following steps:

```
[root@host ~]# mkdir /mnt/data
```

- Add an entry to the `/etc/fstab` file:

```
/dev/vg01/lv01 /mnt/data xfs defaults 1 2
```

- Run **mount /mnt/data** to mount the file system that you just added in **/etc/fstab**.

```
[root@host ~]# mount /mnt/data
```


Removing a Logical Volume

- To remove all logical volume components, perform the following steps:

- **Prepare the file system.**

```
[root@host ~]# umount /mnt/data
```

- **Remove the logical volume.**

```
[root@host ~]# lvremove /dev/vg01/lv01
```

- **Remove the volume group.**

```
[root@host ~]# vgremove vg01
```

- **Remove the physical volumes.**

```
[root@host ~]# pvremove /dev/vdb2 /dev/vdb1
```

Summary of LVM Commands

- Initialize physical volumes.

```
# pvcreate /dev/sdb1 /dev/sdc1 /dev/sdd1  
# pvs  
# pvdisplay
```

- Create volume group "data-vg" with physical volumes.

```
# vgcreate data-vg /dev/sdb1 /dev/sdc1 /dev/sdd1  
# vgs  
# vgdisplay
```

- Create logical volume "data-lv" in volume group.

```
# lvcreate -L 10G -n data-lv data-vg  
# lvs  
# lvdisplay
```

- Use /dev/data-vg/data-lv as a block device.

Summary of LVM Commands

- Removing a logical volume will destroy any data stored on the logical volume.

```
# lvremove /dev/data-vg/data-lv (file system must be unmounted first)
# vgremove data-vg
# pvremove /dev/sdb1 /dev/sdc1 /dev/sdd1
```

- Extending Logical Volumes (no down time):

```
# pvcreate /dev/sde1
# vgextend data-vg /dev/sde1
# lvextend -L +3G /dev/data-vg/data-lv
# xfs_growfs /dev/data-vg/data-lv (update the file system for XFS file systems)

# resize2fs /dev/data-vg/data-lv (update the file system for other file systems)
```

Or:

```
# lvextend -r -L +3G /dev/data-vg/data-lv (extend and update in one step)
```

Summary of LVM Commands

- Removing a logical volume will destroy any data stored on the logical volume.

```
# lvremove /dev/data-vg/data-lv (file system must be unmounted first)
# vgremove data-vg
# pvremove /dev/sdb1 /dev/sdc1 /dev/sdd1
```

- Extending Logical Volumes (no down time):

```
# pvcreate /dev/sde1
# vgextend data-vg /dev/sde1
# lvextend -L +3G /dev/data-vg/data-lv
# xfs_growfs /dev/data-vg/data-lv (update the file system for XFS file systems)

# resize2fs /dev/data-vg/data-lv (update the file system for other file systems)
```

Or:

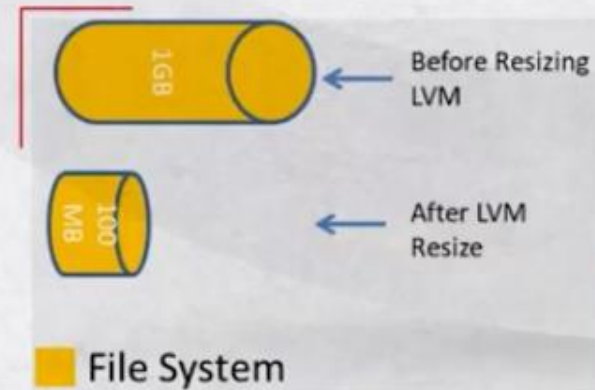
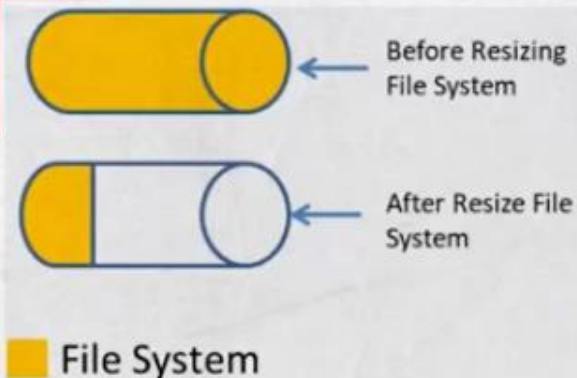
```
# lvextend -r -L +3G /dev/data-vg/data-lv (extend and update in one step)
```


Summary of LVM Commands

- Shrinking a volume group:
 - XFS doesn't support shrinking.

```
No modify flag set, skipping filesystem flush and exiting.  
fsadm: Xfs filesystem shrinking is unsupported.  
/usr/sbin/fsadm failed: 1  
Filesystem resize failed.
```

```
# umount /data  
# resize2fs /dev/data-vg/data-lv 100M  
# e2fsck -f /dev/data-vg/data-lv  
# lvreduce --size -900M /dev/data-vg/data-lv  
# lvreduce --size -r -900M /dev/data-vg/data-lv  
# vgreduce data-vg /dev/sde1      (removes sde1 from data-vg)  
# mount /dev/data-vg/data-lv /data
```



RH134

Implementing Advanced Storage Features



Red Hat | Enterprise Linux

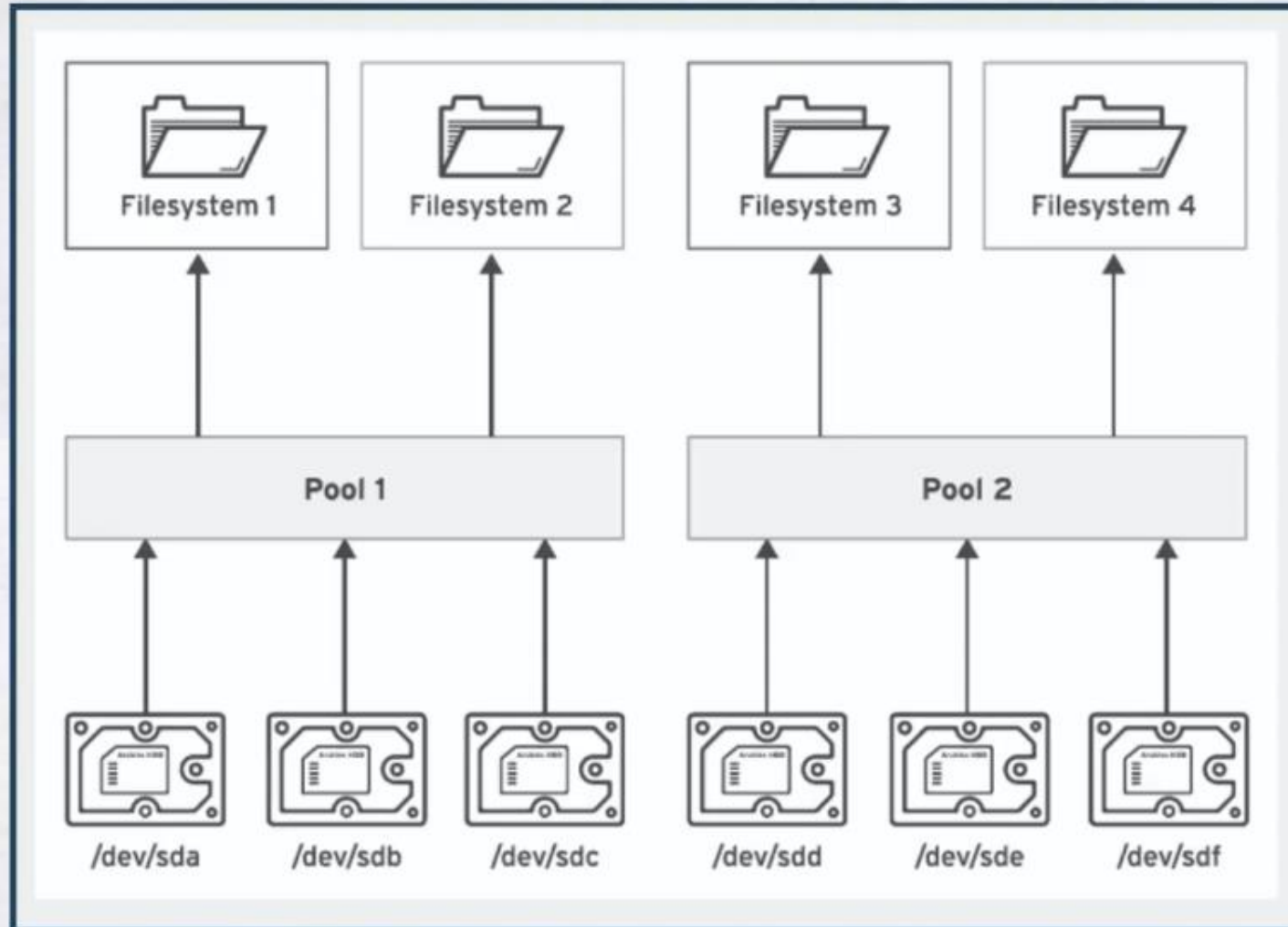
Goal

- Manage storage using the Stratis local storage management system and use the VDO volumes to optimize storage space in use.

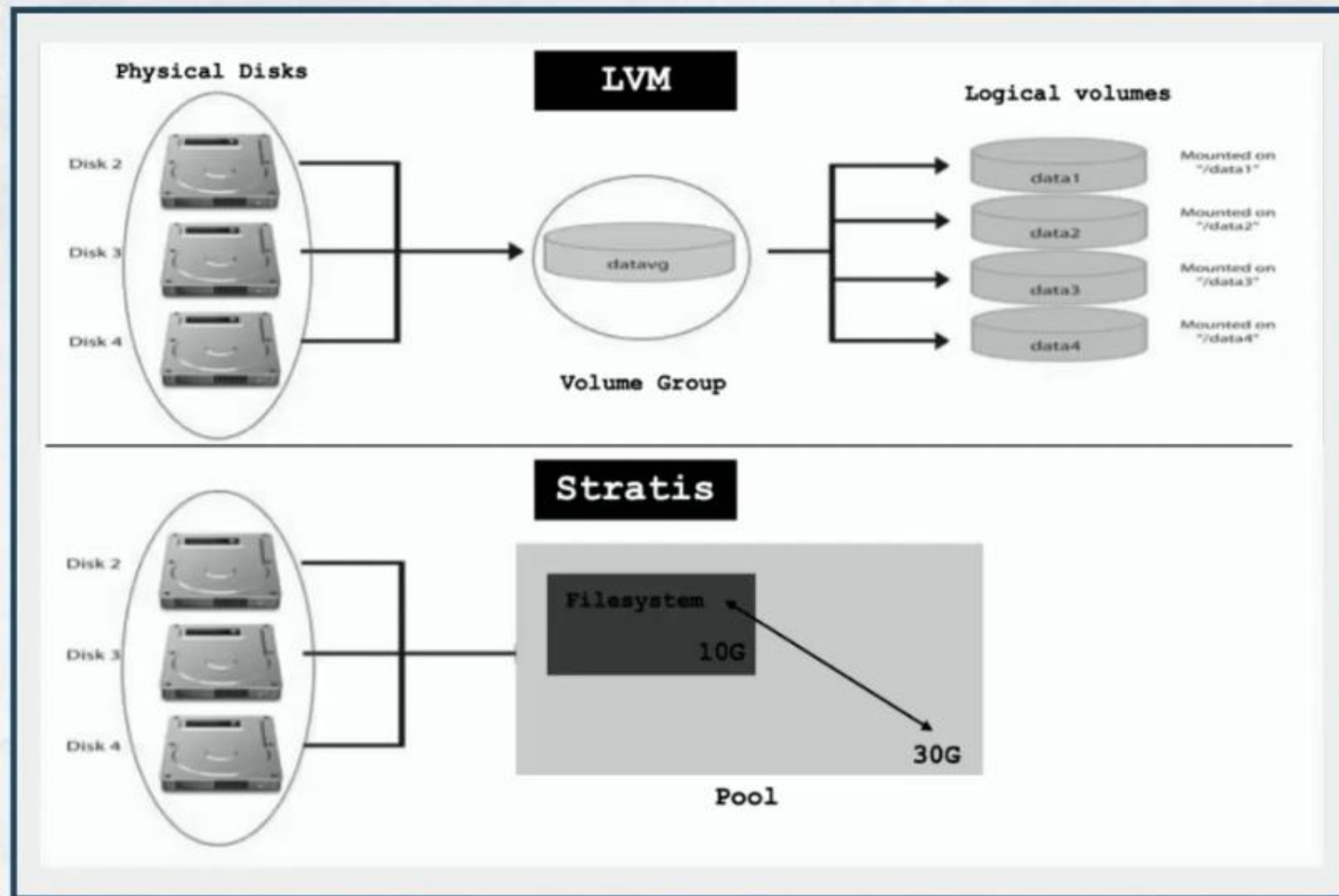
Objectives

- Manage multiple storage layers using Stratis local storage management.
- Optimize the use of storage space by using VDO to compress and deduplicate data on storage devices.

Managing Storage with Stratis



LVM vs. Stratis



Working with Stratis Storage

- To manage file systems with the Stratis storage management solution, install the **stratis-cli** and **stratisd** packages.
- The stratis-cli package provides the stratis command, which sends reconfiguration requests to the stratisd system daemon.
- The stratisd package provides the stratisd service, which handles reconfiguration requests and manages and monitors block devices, pools, and file systems that Stratis uses.

Installing and Enabling Stratis

- To use Stratis, make sure that the software is installed and the **stratisd** service is running.
 - Install stratis-cli and stratisd using the **yum install** command.

```
[root@host ~]# yum install stratis-cli stratisd
```

- Activate the **stratisd** service using the **systemctl** command.

```
[root@host ~]# systemctl enable --now stratisd
```

Assembling Block Storage into Stratis Pools

- The following are common management operations performed using the Stratis storage management solution.
 - Create pools of one or more block devices using the **stratis pool create** command.

```
[root@host ~]# stratis pool create pool1 /dev/vdb
```

- Use the **stratis pool list** command to view the block devices of a pool.

```
[root@host ~]# stratis pool list
```


Assembling Block Storage into Stratis Pools

- The following are common management operations performed using the Stratis storage management solution.
 - Use the **stratis pool add-data** command to add additional block devices to a pool.

```
[root@host ~]# stratis pool add-data pool1 /dev/vdc
```

- Use the **stratis blockdev list** command to view the block devices of a pool.

```
[root@host ~]# stratis blockdev list pool1
```

Managing Stratis File Systems

- Use the **stratis filesystem create** command to create a file system from a pool.

```
[root@host ~]# stratis filesystem create pool1 fs1
```

- Use the **stratis filesystem list** command to view the list of available file systems.

```
[root@host ~]# stratis filesystem list
```

- You can create a snapshot of a Stratis-managed file system with the **stratis filesystem snapshot** command. Snapshots are independent of the source file systems.

```
[root@host ~]# stratis filesystem snapshot pool1 fs1 snapshot1
```


Persistently Mounting Stratis File Systems

- To ensure that the Stratis file systems are persistently mounted, edit **/etc/fstab** and specify the details of the file system. The following command displays the UUID of the file system that you should use in **/etc/fstab** to identify the file system.

```
[root@host ~]# lsblk --output=UUID /stratis/pool1/fs1
```

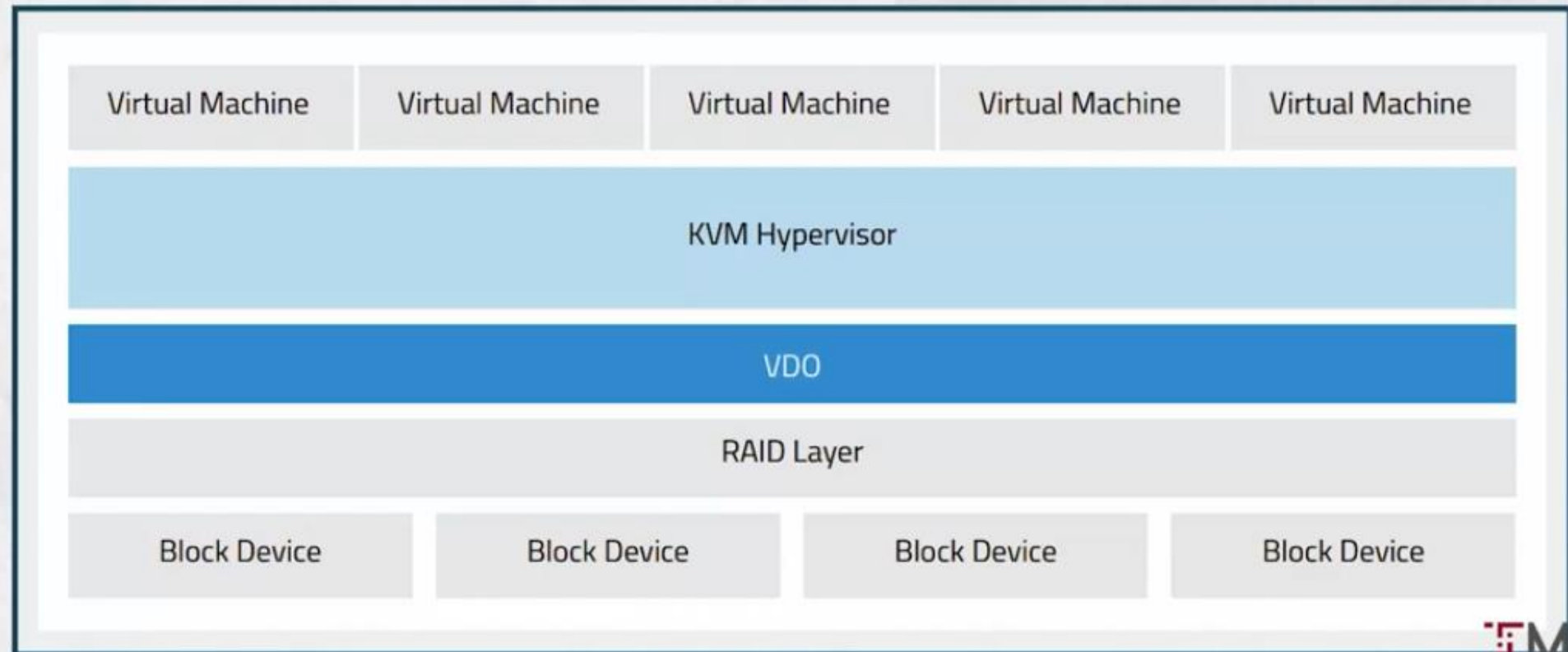
- The following is an example entry in the **/etc/fstab** file to persistently mount a Stratis file system. This example entry is a single long line in the file.

```
UUID=31b9363b-add8-4b46-a4bf-c199cd478c55 /dir1 xfs defaults,xsystemd.requires=stratisd.service 0 0
```

- The **x-systemd.requires=stratisd.service** mount option delays mounting the file system until after **systemd** starts the **stratisd.service** during the boot process.

Compressing and Deduplicating Storage with VDO

- The Virtual Data Optimizer (VDO) is a storage optimization technology developed by Red Hat. It is designed to reduce storage costs and improve efficiency by using data deduplication and compression techniques. VDO operates at the block level and is primarily used for reducing the amount of storage space required for storing data.



➤ **VDO applies three phases to data in the following order to reduce the footprint on storage devices:**

1. Zero Block Elimination:

- In this phase, VDO identifies and eliminates blocks of data that consist entirely of zeros. These blocks are often found in sparse files or during the initial stages of data writes. By eliminating zero blocks, VDO can quickly reclaim unused storage space.

2. Data Compression:

- After zero block elimination, VDO employs data compression techniques to reduce the size of the remaining non-zero data blocks. Data compression algorithms are applied to individual blocks before they are stored on the storage device. This reduces the amount of physical space required to store the data.

3. Deduplication:

- In the deduplication phase, VDO identifies duplicate data blocks and stores only one instance of each unique block. This process is crucial for eliminating redundant copies of data that may be present across different files or versions. Deduplication contributes significantly to reducing the overall storage footprint.

Virtual Data Optimizer (VDO)

➤ Enabling VDO

- Install the **vdo** and **kmod-kvdo** packages to enable VDO in the system.

```
[root@host ~]# yum install vdo kmod-kvdo
```

➤ Creating a VDO Volume

- To create a VDO volume, run the **vdo create** command.

```
[root@host ~]# vdo create --name=vdo1 --device=/dev/vdd --vdoLogicalSize=50G
```

➤ Analyzing a VDO Volume

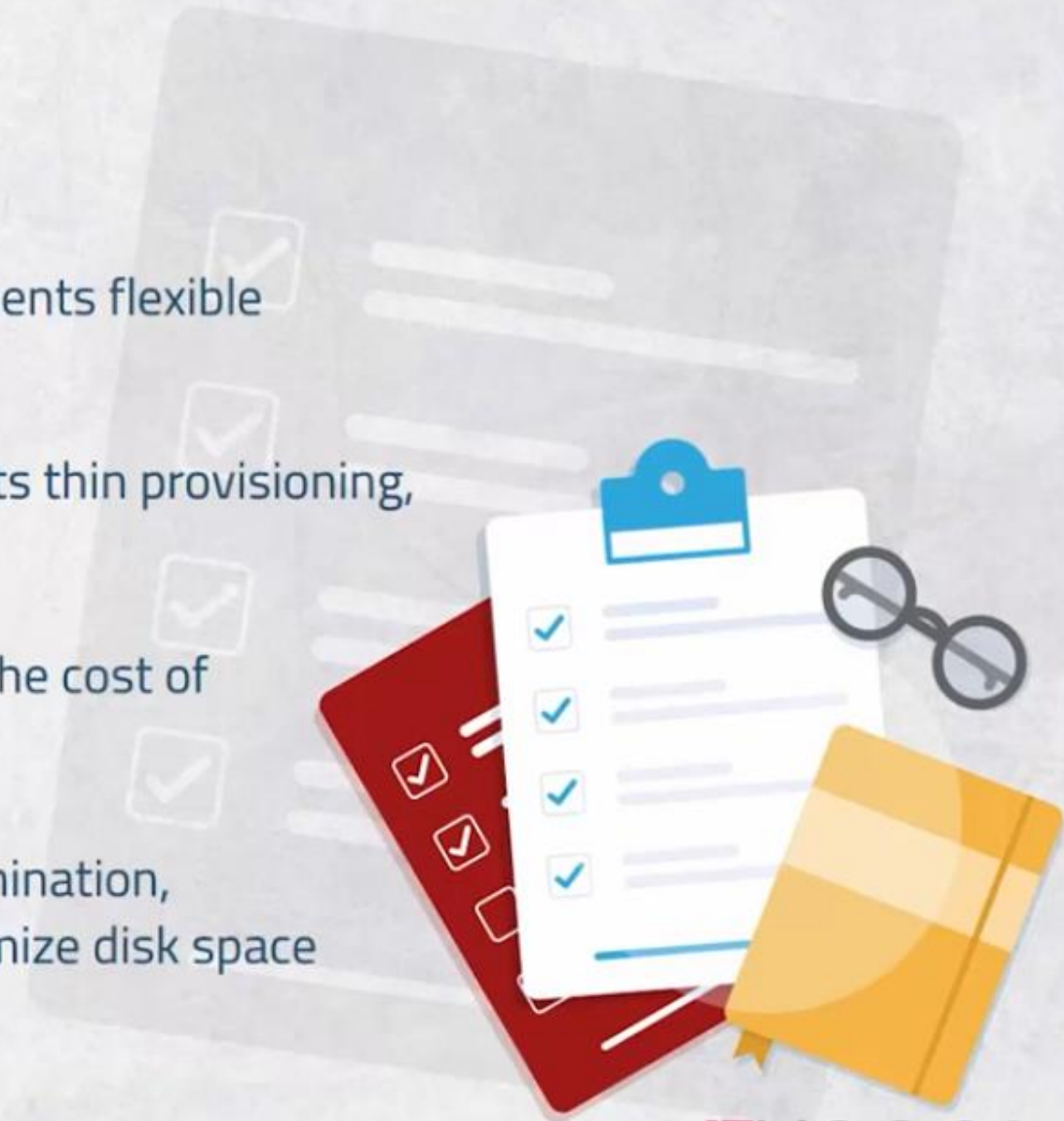
- To analyze a VDO volume, run the **vdo status** command.

```
[root@host ~]# vdo status --name=vdo1
```


Summary

In this chapter, you learned:

- The Stratis storage management solution implements flexible file systems that grow dynamically with data.
- The Stratis storage management solution supports thin provisioning, snapshotting, and monitoring.
- The Virtual Data Optimizer (VDO) aims to reduce the cost of data storage.
- The Virtual Data Optimizer applies zero-block elimination, data deduplication, and data compression to optimize disk space efficiency.



RH134

Accessing Network-Attached Storage



Red Hat | Enterprise Linux

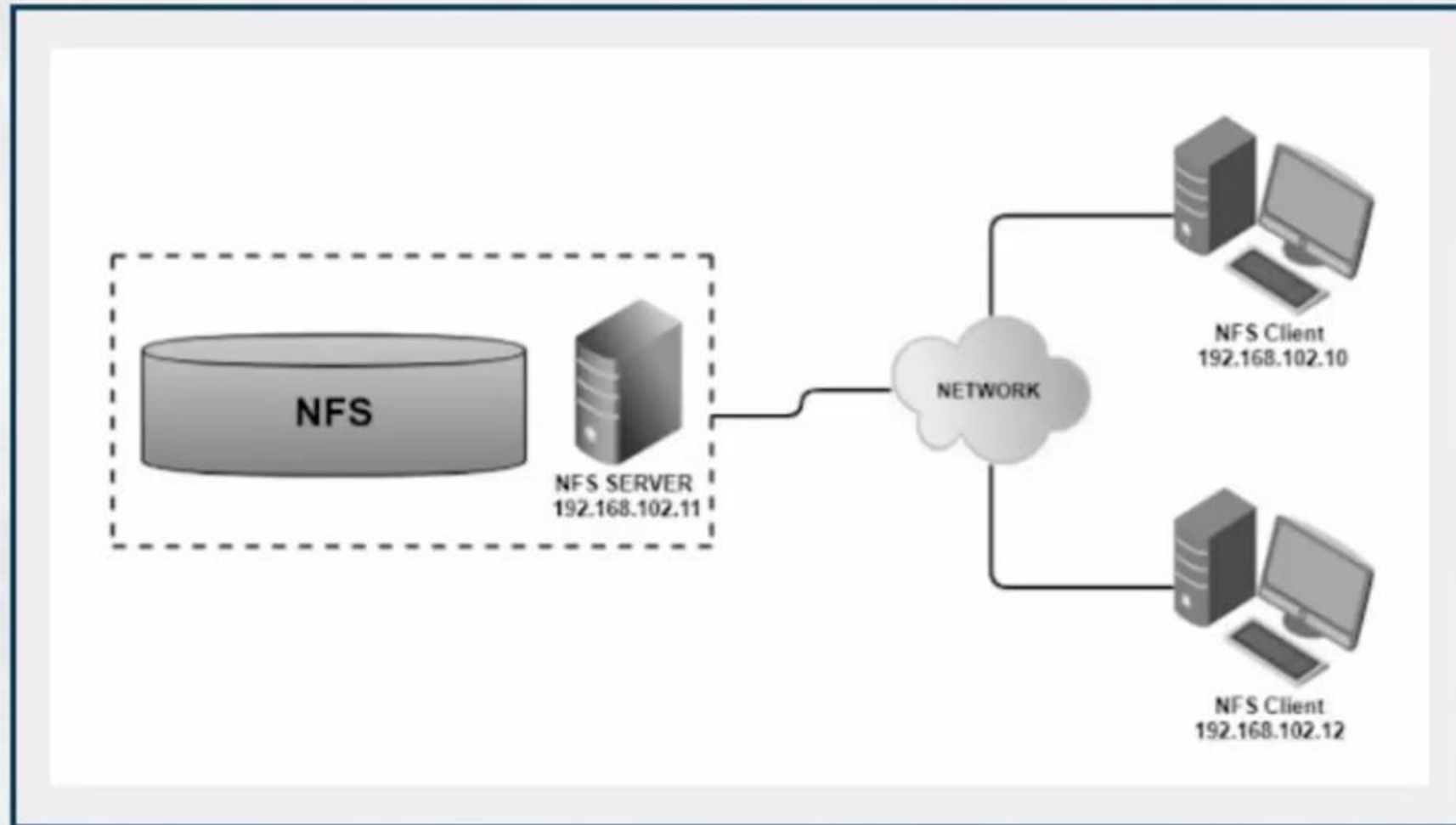
Goal

- Access network-attached storage using the NFS protocol.

Objectives

- Mount, use, and unmount an NFS export from the command line and at boot time.
- Configure the automounter with direct and indirect maps to automatically mount an NFS file system on demand, and unmount it when it is no longer in use.

Accessing Network Storage with Network File System (NFS)



Setup and Configure NFS Mounts on Linux Server

➤ To setup NFS mounts, we'll be needing at least two Linux/Unix machines.

- ✓ NFS Server
- ✓ NFS Client

Installing NFS Server and NFS Client

➤ We need to install NFS packages on our NFS Server as well as on NFS Client machine.

```
[root@master ~]# yum install nfs-utils  
[root@master ~]# systemctl start nfs-server  
[root@master ~]# systemctl enable nfs-server  
[root@master ~]# systemctl stop firewalld
```

Setting Up the NFS Server

```
[root@master ~]# mkdir /data
[root@master ~]# vim /etc/exports
/data      192.168.1.0/24(ro)
/data      192.168.1.101(rw)
[root@master ~]# systemctl restart nfs-server    (terminates all open sessions)
[root@master ~]# exportfs -r                    (re-read the config file without termination of open sessions)
[root@master ~]# exportfs -f                    (force re-read the config file)
```


Setting Up the NFS Client

```
[root@client ~]# showmount -e 192.168.1.100
[root@client ~]# mount 192.168.1.100:/data /root/data
[root@client ~]# df -hT (nfs4)
[root@client ~]# mount -t nfs 192.168.1.100:/data /root/data
[root@client ~]# vim /etc/fstab
```

```
192.168.1.100:/data    /root/data    nfs    defaults    0    0
```

Automounting Network-Attached Storage

Mounting NFS Shares with the Automounter

- The automounter is a service (**autofs**) that automatically mounts NFS shares "on-demand," and will automatically unmount NFS shares when they are no longer being used.

Automounter Benefits

- Users do not need to have root privileges to run the **mount** and **umount** commands.
- NFS shares configured in the automounter are available to all users on the machine, subject to access permissions.
- NFS shares are not permanently connected like entries in **/etc/fstab**, freeing network and system resources.
- The automounter is configured on the client side; no server-side configuration is required.

Steps for Automounting NAS Using Autofs

1. Install the autofs package.

- Ensure Autofs is installed on your RHEL 9 system

```
[user@host ~]# dnf install autofs
```

2. Add a master map file to /etc/auto.master.d. This file identifies the base directory used for mount points and identifies the mapping file used for creating the automounts.

```
[user@host ~]# vim /etc/auto.master.d/demo.autofs
```

- Add the master map entry, in this case, for indirectly mapped mounts:

```
/shares /etc/auto.demo
```


Steps for Automounting NAS Using Autofs

3. Create the mapping files. Each mapping file identifies the mount point, mount options, and source location to mount for a set of automounts.

```
[user@host ~]# vim /etc/auto.demo
```

- The mapping file-naming convention is /etc/auto.name, where name reflects the content of the map

```
work -rw,sync serverb:/shares/work
```

4. Start and enable the automounter service.

- Use systemctl to start and enable the autofs service.

```
[user@host ~]$ sudo systemctl enable --now autofs
```

Direct Maps

- Direct maps are used to map an NFS share to an existing absolute path mount point.
- To use directly mapped mount points, the master map file might appear as follows:

```
/- /etc/auto.direct
```

- All direct map entries use `/-` as the base directory. In this case, the mapping file that contains the mount details is **`/etc/auto.direct`**.
- The content for the `/etc/auto.direct` file might appear as follows:

```
/mnt/docs -rw,sync serverb:/shares/docs
```

- The mount point (or key) is always an absolute path. The rest of the mapping file uses the same structure.
- In this example the **`/mnt`** directory exists, and it is not managed by **autofs**. The full directory **`/mnt/docs`** will be created and removed automatically by the **autofs** service.

Indirect Wildcard Maps

- When an NFS server exports multiple subdirectories within a directory, then the automounter can be configured to access any one of those subdirectories using a single mapping entry.
- Continuing the previous example, if `serverb:/shares` exports two or more subdirectories and they are accessible using the same mount options, then the content for the **/etc/auto.demo** file might appear as follows:

```
* -rw,sync serverb:/shares/&
```

- The mount point (or key) is an asterisk character (*), and the subdirectory on the source location is an ampersand character (&). Everything else in the entry is the same.
- When a user attempts to access **/shares/work**, the key * (which is **work** in this example) replaces the ampersand in the source location and `serverb:/shares/` is mounted. As with the indirect example, the **work** directory is created and removed automatically by autofs.

Summary

In this chapter, you learned how to:

- Mount and unmount an NFS export from the command line.
- Configure an NFS export to automatically mount at startup.
- Configure the automounter with direct and indirect maps, and describe their differences.

